[B 书] TPMOS_TEXTBOOK [GRAD 学]

The Definitive Guide to the Mono-OS (基于件的系统指南)

Version 03.01 (Code-Complete Edition)

Softness wins. The empty center of the flexbox holds ten thousand things. [ZEN 禅]

[DOC 单] Table of Contents (目录)

Version 03.01 (Code-Complete Edition)

Part I: Foundations

- The Soul of a Piece (Philosophy)
- The Filesystem is the Database
- The 12-Step Pipeline

Part II: Development

- Muscles & Brains (Ops & Modules)
- The App Factory
- PAL: The Assembly Language of TPMOS

Part III: Flagship Applications

- fuzz-op & op-ed: The Flagship Apps
- GL-OS: Beyond ASCII

Part IV: Advanced Systems

- The Guardians (Testing & Simulation)
- The Great Expansion (AI, LSR, & P2P)
- The Infinite Loop (Recursive Forge)
- The Simulation Theater

Part V: Vision & Future

- Piecemark Labs & The Sovereign Venture
- The Soul Pen & The Multiverse
- Cross-Platform TPMOS
- Common Pitfalls & Debugging Guide
- Glossary
- Quiz
- Answer Key
- CH1: Philosophy

- CH2: Filesystem
- CH3: Pipeline
- CH4: Ops & Modules
- CH5: App Factory
- CH6: PAL
- CH7: fuzz-op & op-ed
- CH8: GL-OS
- CH9: Testing
- CH10: P2P-NET
- CH11: Recursive Forge
- CH12: Simulation
- CH13: Business
- CH14: Soul Pen
- CH15: Cross-Platform
- CH16: Pitfalls
- Return to Main Index

[DNA 命] Chapter 1: The Soul of a Piece

The True Piece Method (TPM) is the governing philosophy of this entire OS. In TPMOS, we don't think in terms of objects or classes; we think in terms of Pieces. [PIECE 砖]

[BLD 建] The PMO Hierarchy

The system follows a strict thought priority. If you violate this, the "Mirror" will crack! [MIRROR 镜]

1. PIECE (The Atomic Unit / Soul) [PIECE 砖]
   - Everything is a Piece. A button, a player, a map, or a galaxy.
   - **Rule:** A Piece owns its state *exclusively*.
   - **File:** `piece.pdl` (DNA) + `state.txt` (Mirror).

2. MODULE (The Logic Agent / Brain) [MIND 脑]
   - The Module is the "Manager". It listens to the user and decides what the Pieces should do.
   - **Mandate:** It should be a "Thin Brain". It doesn't do the heavy lifting; it delegates.

3. OS / CHTPM (The Theater / View) [VIEW 戏]
   - The OS is the stage where Pieces perform. It handles the UI, the layouts, and the "Magic" (variable substitution).

[DNA 命] Anatomy of Piece DNA (`.pdl`)

Every Piece has a `.pdl` file. This is its blueprint. Think of it as the code that defines what a Piece is and what it can do.

```
<piece_id>my_hero</piece_id>
<traits>
    <trait>movable</trait>
    <trait>auditable</trait>
</traits>
<methods>
    <method id="move_north" cmd="./move_entity.+x north" />
    <method id="say_hello" cmd="echo 'Hello World!' > last_response.txt" />
</methods>
```

[MIRROR 镜] The Mirror (`state.txt`)

While DNA defines the potential, the Mirror defines the now. It is a flat text file for high-speed reading.

- `x=10`
- `y=5`
- `health=100`
- `status=idle`

[TIPS 明] **Pro Tip:** "If it's not in a file, it's a lie." We never trust memory. If a process dies, the file remains. Reality persists.

[ZEN 禅] The Zen of TPM

- **Data Sovereignty:** No piece touches another's files without permission.
- **Auditability:** Every major change is logged to the `master_ledger.txt`.
- **Recursive Reality:** A Piece can contain other Pieces (Inception style! [REC 归]). This is known as the **Scale-Free Container Model**.

The Scale-Free Container Model

In TPMOS, directory containment is the authoritative source of truth.

- **Maps are Containers:** A map is just a Piece that contains other Pieces (Player, NPCs, Trees).
- **Pieces are Containers:** A Player Piece can contain a "Bag" Piece, which contains a "Crayon Box" Piece, which contains "Crayon" Pieces.

- **Unified Rule:** The engine treats all containers uniformly. Whether it's a Galaxy containing Solar Systems or a Chest containing Gold, the logic is identical. [COSMOS 宇][$ 钱]

[BLD 馆] Scholar's Corner: The "Mirror of Tomokazu"

There is a legendary anecdote among TPMOS engineers known as the "Mirror of Tomokazu." Early in development, a rogue developer tried to optimize the system by keeping a player's HP in a global variable instead of writing it to `state.txt`. During a stress test, the process crashed. When the system rebooted, the player had 0 HP and was "permanently dead," despite having been full health seconds before. This disaster led to the founding mandate: "If it's not in a file, it's a lie." From that day on, every piece of reality had to be mirrored in a file, ensuring immortality through persistence. [IMM 烛]

[DOC 记] Study Questions

1. What are the three layers of the PMO hierarchy?
2. What is the difference between a Piece's "DNA" (.pdl) and its "Mirror" (state.txt)?
3. Why does TPMOS avoid using in-memory variables for long-term state?
4. True or False: A galaxy and a button can both be considered "Pieces" in TPMOS.

[Return to Index](INDEX.md)

Chapter 2: The Filesystem is the Database

In a traditional OS, the filesystem is just for storage. In TPMOS, the filesystem is the system. [BLD 馆]

[MAP 图] The Project Map

Everything has a place, and there is a place for everything. The root of the world is defined in `pieces/locations/location_kvp`.

`location_kvp`: The Compass

This file tells the system where to find the "Ground". If you move the project, you only change this file.

- `project_root=/home/user/TPMOS/`
- `pieces_dir=pieces/`
- `apps_dir=pieces/apps/`

The Directory Hierarchy

Let's look at how a project is structured. It's like a nested doll.

```
/projects/my_game/
├── project.pdl         # The "World DNA"
├── manager/            # The "World Brain" (.+x binaries)
├── maps/               # The "Geography"
└── pieces/             # The "Inhabitants"
    ├── hero/
    │   ├── piece.pdl   # Hero DNA
    │   └── state.txt   # Hero Stats
    └── slime_01/
        ├── piece.pdl   # Slime DNA
        └── state.txt   # Slime Stats
```

File-Based IPC (The Mailbox)

How do two programs talk to each other if they don't share memory? They write notes!

1. The History Buffer: `keyboard/history.txt`. The input driver writes `A`, the Parser reads `A`.
2. The Pulse: `frame_changed.txt`. When a renderer finishes a frame, it "pulses" this file. Other programs see the pulse and wake up.
3. The Response: `last_response.txt`. When an Op runs, it writes its output here so the UI can show it to the user.

[SEC 安] CPU Safety & The `stat()` Guard

Reading files constantly can be heavy. To stay fast, we use the Stat-First Pattern. [SEC 安]

- **Logic:** Don't open the file unless `stat()` tells you the "Modified Time" (mtime) has changed.
- **Result:** 0% CPU usage when nothing is happening. TPMOS is a "Green" OS!

Code Example: The `stat()` Guard Pattern

This is the most important pattern in TPMOS. Every module uses it:

```
#include <sys/stat.h>

long last_pos = 0;
struct stat st;

// Only read the file if its size has changed
if (stat(history_path, &st) == 0) {
    if (st.st_size > last_pos) {
        // New input available - process it
        FILE *hf = fopen(history_path, "r");
        if (hf) {
            fseek(hf, last_pos, SEEK_SET);
            int key;
            while (fscanf(hf, "%d", &key) == 1) {
                process_key(key);
            }
            last_pos = ftell(hf);
            fclose(hf);
        }
    } else if (st.st_size < last_pos) {
        // File was truncated - reset position
        last_pos = 0;
    }
}
```

This pattern ensures 0% CPU usage when idle. The module sleeps for 16ms between checks, never spinning wastefully.

Code Example: Parsing `location_kvp`

Every TPMOS binary starts by resolving its paths from `location_kvp`. Here's how it works:

```
char project_root[4096] = ".";

void resolve_paths() {
    FILE *kvp = fopen("pieces/locations/location_kvp", "r");
    if (!kvp) return;

    char line[1024];
    while (fgets(line, sizeof(line), kvp)) {
        char *eq = strchr(line, '=');
        if (eq) {
            *eq = '\0';
            char *key = trim_str(line);
            char *val = trim_str(eq + 1);
            if (strcmp(key, "project_root") == 0) {
                snprintf(project_root, sizeof(project_root), "%s", val);
            }
        }
    }
    fclose(kvp);
}
```

The `trim_str()` helper removes whitespace from keys and values. This ensures paths are clean even if the file has trailing newlines.

Code Example: Reading Piece State

Once paths are resolved, reading a Piece's state is straightforward:

```
int get_state_int(const char* piece_id, const char* key) {
    char path[4096];
    snprintf(path, sizeof(path), "%s/projects/%s/pieces/%s/state.txt",
             project_root, current_project, piece_id);

    FILE *f = fopen(path, "r");
    if (!f) return -1;

    char line[256];
    int val = -1;
    while (fgets(line, sizeof(line), f)) {
        char *eq = strchr(line, '=');
        if (eq && strncmp(line, key, strlen(key)) == 0) {
            val = atoi(trim_str(eq + 1));
            break;
        }
    }
    fclose(f);
    return val;
}

// Usage:
int player_x = get_state_int("player", "pos_x");
int player_y = get_state_int("player", "pos_y");
```

Example: `state.txt` Format

A Piece's state file is a simple key-value list:

```
name=Zombie
type=zombie
pos_x=9
pos_y=2
pos_z=0
on_map=1
behavior=aggressive
speed=1
map_id=map_01_z0.txt
```

No JSON, no SQLite - just plaintext. This is Data Sovereignty in action: any tool can read and modify state, from `cat` to a C binary to an AI agent.

Windows Compatibility Notes

TPMOS is designed to be cross-platform. Here's what changes on Windows:

Path Differences

- Linux/macOS: Forward slashes `/` (e.g., `pieces/world/map_01/`)
- Windows: Backslashes `\` (e.g., `pieces\world\map_01\`)
- Solution: Use `build_path_malloc()` helper which handles both:

```c
char build_path_malloc(const char rel) {
size_t sz = strlen(project_root) + strlen(rel) + 2;
char p = (char)malloc(sz);
if (p) snprintf(p, sz, "%s/%s", project_root, rel);
return p;
}
```

File API Differences

Linux/macOS	Windows Equivalent	Notes
`access(path, F_OK)`	`_access(path, 0)`	Use `#ifdef _WIN32`
`stat(path, &st)`	`stat(path, &st)`	Works on both (MSVCRT)
`getcwd(buf, sz)`	`_getcwd(buf, sz)`	Same header `<direct.h>`
`usleep(us)`	`Sleep(us/1000)`	`<windows.h>` vs `<unistd.h>`

Cross-Platform Pattern

```
#ifdef _WIN32
    #include <windows.h>
    #include <direct.h>
    #include <io.h>
    #define usleep(us) Sleep((us)/1000)
    #define access _access
    #define F_OK 0
#else
    #include <unistd.h>
    #include <sys/stat.h>
#endif
```

Binary Convention

- Linux/macOS: `.+x` extension (e.g., `move_entity.+x`)
- Windows: `.exe` extension (e.g., `move_entity.exe`)
- The compile scripts (`compile_all.sh` vs `compile_all.psl`) handle this automatically.

[WARN 警] Common Pitfalls

Pitfall 1: Hardcoded Paths

Symptom: Binary works on your machine but fails after moving the project folder.

```
// [FAIL 错] BAD: Hardcoded absolute path
FILE *f = fopen("/home/user/TPMOS/projects/fuzz-op/pieces/zombie/state.txt", "r");

// [OK 是] GOOD: Resolve from location_kvp
resolve_paths();
char path[4096];
snprintf(path, sizeof(path), "%s/projects/%s/pieces/zombie/state.txt",
         project_root, current_project);
```

Fix: Always call `resolve_paths()` at startup and use `project_root` variable.

Pitfall 2: Forgetting to Update `location_kvp` After Moving

Symptom: Entire project "disappears" - no pieces, no maps, nothing loads.

Cause: `location_kvp` still points to the old directory.

Fix: Update the `project_root=` line in `pieces/locations/location_kvp` to the new path. The entire world will "wake up" instantly.

Pitfall 3: `stat()` Race Conditions

Symptom: Input is occasionally missed or processed twice.

Cause: File size check and read aren't atomic - another process can write between them.

Fix: Always use `fseek(hf, last_pos, SEEK_SET)` before reading, and update `last_pos = ftell(hf)` after. If `st.st_size < last_pos`, reset to 0 (file was truncated).

Pitfall 4: Reading State Without Checking File Exists

Symptom: Crash or garbage values when reading a Piece that doesn't exist.

Fix: Always check `fopen()` return value:

```
FILE *f = fopen(path, "r");
if (!f) return -1;  // Piece doesn't exist yet
```

[BLD 馆] Scholar's Corner: The "Lost Map of Lunar Streetrace"

Legend has it that a project was once "lost" during a massive server migration. For three days, the `lsr` simulation was invisible. Then, a developer realized that only the `location_kvp` file had been corrupted. The entire world—its companies, its bots, its history—was still there in the `projects/` folder. By writing a single line back into `location_kvp`, the entire universe "woke up" as if no time had passed. This proved that in TPMOS, "Geography is Destiny." If the files exist, the world is never truly lost. [MAP 图][闪]

[DOC 记] Study Questions

1. What is the role of the `location_kvp` file?
2. Explain File-Based IPC in your own words.
3. How does the "Stat-First Pattern" improve CPU efficiency?
4. Imagine: You want to move your TPMOS project to a different drive. Which file do you need to update to ensure everything still works?

[Return to Index](INDEX.md)

[PULSE 心] Chapter 3: The 12-Step Pulse

How does a single keypress turn into a pixel on your screen? It's a 12-step journey through the system's "Circulatory System".

[SPRINT 跑] The 12-Step Sprint

Step 1: [CPU 键] INPUT

`keyboard_input.+x` (the Muscle) catches your key and scribbles it into `pieces/keyboard/history.txt`.

```
// keyboard_input_linux.c - Raw terminal input
struct termios raw = termios;
raw.c_lflag &= ~(ICANON | ECHO);  // Disable line buffering
tcsetattr(STDIN_FILENO, TCSANOW, &raw);

int key = getchar();  // Read single keystroke
fprintf(history_file, "%d\n", key);  // Write to history buffer
```

Step 2-3: [MAP 图] ROUTING & [PR 喇] RELAY

`chtpm_parser.c` (the OS) reads the key. It uses W-first (World-First) resolution, checking `world_xx/map_xx/` paths before legacy fallbacks.

```
// chtpm_parser.c - W-first path resolution
// Check project-specific world first
snprintf(path, sizeof(path), "%s/projects/%s/pieces/world/",
         project_root, current_project);
if (access(path, F_OK) == 0) {
    // Use project-specific world
} else {
    // Fallback to global pieces/world/
}
```

If the user is "interacting" with an app, the Parser throws the key into the app's own history buffer (e.g., `player_app/history.txt`).

[MODE 模式] The Dual Execution Model

Once the key is relayed, the system enters one of two "modes":

- **[FAST 快] Mode 1: Realtime Input (Direct C)**
- **Latency:** ~16ms (Instant).
- **Action:** Movement (WASD), Menu Navigation.
- **Path:** Module calls Muscle Op directly. No scripting involved.
- ** Mode 2: Event Scripts (PAL Scripting)**
- **Latency:** ~50-100ms.
- **Action:** NPC Dialogue, Quests, AI triggers.
- **Path:** Module calls `prisc+x` -> executes `event.asm` -> calls Muscle Ops.

Step 4: [MIND 脑] TICK

The App Module (the Brain) wakes up! It sees the new key in its history and decides which Mode to use.

Step 5-6: TRAIT/OP & [PIECE 砖] SOVEREIGNTY

The Brain says, "The user pressed 'UP'. Call the `move_entity.+x` Muscle!" The Muscle checks boundaries and updates the Piece's `state.txt`.

```
// Module calls Op via fork()/exec()
char* trait = build_path_malloc("pieces/apps/playrm/ops/+x/move_entity.+x");
pid_t p = fork();
if (p == 0) {
    freopen("/dev/null", "w", stdout);
    freopen("/dev/null", "w", stderr);
    execl(trait, trait, active_target_id, dir_str, NULL);
    exit(1);
} else {
    waitpid(p, NULL, 0);  // Wait for Op to complete
}
free(trait);
```

Why `fork()/exec()` instead of `system()`? Because `system()` spawns a subshell that can become orphaned on Ctrl+C. With `fork()/exec()/waitpid()`, we have full control over the child lifecycle - no zombie processes!

Step 7: [MIRROR 镜] MIRROR SYNC

The Muscle flushes the new state to disk. Reality is now officially updated.

Step 8: STAGE

`render_map.+x` (the Stage Producer) sees the change. It reads ALL the Pieces on the map and draws a "View" (`view.txt`).

```
// render_map.c - Read all pieces, produce view.txt
DIR *dir = opendir(pieces_dir);
struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    if (entry->d_name[0] == '.') continue;
    // Read each piece's state.txt
    // Check if piece is on current map
    // Write to view.txt with position and character
}
closedir(dir);
```

Step 9-12: SYNC → [GL 艺] COMPOSITION → RENDER → [TERM 视] DISPLAY

The Stage Producer updates the app's general state. The Parser takes `view.txt` and swaps out all variables (like `${player_health}`). The final frame is written to `current_frame.txt`, and the Renderer (ASCII or GL) blasts it onto your monitor!

Pipeline Architecture Diagram

```
┌──────────────────────────┐
│   KEYBOARD    │ Step 1: Write key to history.txt
│   (Muscle)    │
└──────────────┬───────────┘
       │
       ▼
┌──────────────────────────┐
│    PARSER     │ Steps 2-3: Route key to app history
│    (OS)       │ W-first path resolution
└──────────────┬───────────┘
       │
       ▼
┌──────────────────────────┐
│    MODULE     │ Step 4: Read history, decide action
│    (Brain)    │
└──────────────┬───────────┘
       │
       ▼
┌──────────────────────────┐
│      OP       │ Steps 5-6: Execute action, update state
│   (Muscle)    │ fork()/exec()/waitpid()
└──────────────┬───────────┘
       │
       ▼
┌──────────────────────────┐
│   STATE.TXT   │ Step 7: Mirror sync (reality updated)
│   (Disk)      │
└──────────────┬───────────┘
       │
       ▼
┌──────────────────────────┐
│   RENDER_MAP  │ Step 8: Read all pieces, produce view
│   (Stage)     │
└──────────────┬───────────┘
       │
       ▼
┌──────────────────────────┐
│   PARSER      │ Steps 9-11: Compose frame with variables
│   (Theater)   │
└──────────────┬───────────┘
       │
       ▼
┌──────────────────────────┐
│   RENDERER    │ Step 12: Display on screen
│   (ASCII/GL)  │
└──────────────────────────┘
```

The Pulse Frequency

This entire 12-step loop happens in milliseconds. It is the "Heartbeat" of TPMOS. [PULSE 心]

[WARN 警] **Warning:** If any step takes too long (e.g., a slow script), the pulse "stutters". This is why we keep Muscles small and fast! [FAST 快]

Windows Note: `fork()/exec()` → `CreateProcess()`

On Windows, the POSIX `fork()/exec()` pattern doesn't exist. Instead, we use `CreateProcess()`:

```
#ifdef _WIN32
    STARTUPINFO si = {0};
    PROCESS_INFORMATION pi;
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESTDHANDLES;
    // Hide window, redirect output
    CreateProcess(NULL, cmd_line, NULL, NULL, FALSE,
                  CREATE_NO_WINDOW, NULL, NULL, &si, &pi);
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
#else
    // fork()/exec()/waitpid() on Linux/macOS
    pid_t p = fork();
    if (p == 0) { execl(...); exit(1); }
    else waitpid(p, NULL, 0);
#endif
```

The dual-execution model still works identically - just the spawn mechanism differs. The compile scripts (`compile_all.sh` vs `compile_all.psl`) handle building the correct version for each platform.

[WARN 警] Common Pitfalls

Pitfall 1: Using `system()` Instead of `fork()/exec()`

Symptom: After pressing Ctrl+C to quit, orphaned processes keep running and consuming CPU.

Cause: `system()` spawns a subshell that becomes orphaned on signal.

Fix: Always use `fork()/exec()/waitpid()`:

```
// [FAIL 错] BAD
system("move_entity.+x player up");

// [OK 是] GOOD
pid_t p = fork();
if (p == 0) { execl(...); exit(1); }
else waitpid(p, NULL, 0);
```

Pitfall 2: Forgetting `waitpid()` After `fork()`

Symptom: Zombie processes accumulate (ironically, actual zombie processes!).

Cause: Parent doesn't reap child exit status.

Fix: Always call `waitpid(pid, &status, 0)` in the parent branch.

Pitfall 3: Not Hitting `frame_changed.txt` After State Changes

Symptom: Piece state updates but screen doesn't refresh.

Cause: Renderer only checks for changes when `frame_changed.txt` is modified.

Fix: Call `hit_frame_marker()` after any state change:

```
FILE *f = fopen("pieces/display/frame_changed.txt", "a");
if (f) { fprintf(f, "X\n"); fclose(f); }
```

Pitfall 4: Blocking the Pipeline with Slow Ops

Symptom: System "stutters" or feels laggy.

Cause: An op or script takes too long, delaying the next pulse.

Fix: Keep ops under 16ms. If heavy work is needed, do it in a background process and let the pipeline continue.

[BLD 馆] Scholar's Corner: The "Pulse That Never Ended"

During an early experiment with recursive PAL scripts (Chapter 7), a developer accidentally created a loop where a bot would "move north" every time the frame changed. Because the pulse was so fast, the bot moved through 10,000 maps in under a minute! The developer tried to kill the process, but every time a new frame rendered, a new bot was spawned. This became known as "The Pulse That Never Ended." It taught us the critical importance of the Kill All script and the value of throttling (Chapter 4). [PULSE 心][INF 无]

[DOC 记] Study Questions

1. Describe the journey of a keypress from Step 1 to Step 12.
2. What is the role of the "Stage Producer" in the 12-step pipeline?
3. Why is "Mirror Sync" (Step 7) so critical before the "Stage" (Step 8) begins?
4. If the renderer "stutters," which step in the pipeline is most likely the culprit?

[Return to Index](INDEX.md)

Chapter 4: Muscles & Brains (Ops & Modules)

In TPMOS, we separate Thinking from Doing. This is the key to a stable, CPU-safe system. [MIND 脑]

[MIND 脑] The Brain (The Module)

The Module is a long-running background process (the "Brain"). Its job is to listen for user input and make decisions.

- **File:** Usually named `*_module.c` or `*_manager.c`.
- **Job:** Polls the `history.txt` buffer and updates the `state.txt` mirror.

[SEC 安] The CPU-Safe Mandate

Because Modules run in the background, they must be "polite" to the CPU. We follow the CPU-Safe Template:

1. Throttling: If the app isn't on screen, the Module sleeps for 100ms. If it is on screen, it sleeps for 16ms (60 FPS). [IDLE 睡]
2. Stat-First: Don't read the history file unless its size has changed (`stat()`).
3. Signal Handling: Always catch `SIGINT` (Ctrl+C) so you can clean up your children. [CLEAN 洁]

[SEC 安] The Network & Path Safety Mandates (New April 2026)

As TPMOS evolves, we've added two critical mandates for system stability:

1. Dynamic Path Mandate: NEVER hardcode project paths. Always resolve the `project_root` from `pieces/locations/location_kvp`. This ensures your code works even if the folder is moved or versioned. [MAP 图]
2. Network Safety Mandate: Ghost listeners cause crashes! All network-aware modules must ensure their ports (default 8000-8010) are released on exit. The `kill_all.sh` script is the surgical tool for this. 释放端口!

The Muscle (The Op)

The Op is a short-lived binary (the "Muscle"). It does one thing and then exits.

- **File:** Usually ends in `.+x` (e.g., `move_entity.+x`).
- **Job:** Performs a specific action like moving a player or placing a tile.

[NO 禁] Why no `system()`?

We never use the `system()` command in C. Why? Because it's hard to kill! Instead, we use `fork()` and `exec()`.

- **Fork:** Create a child process.
- **Exec:** Turn that child into the Muscle.
- **Wait:** The Brain waits for the Muscle to finish.

Code Comparison: `system()` vs `fork()/exec()`

```
// [FAIL 错] BAD: Can't kill, becomes orphaned on Ctrl+C
system("pieces/apps/playrm/ops/+x/move_entity.+x player up");

// [OK 是] GOOD: Full control, always reaped
char* op_path = build_path_malloc("pieces/apps/playrm/ops/+x/move_entity.+x");
pid_t pid = fork();
if (pid == 0) {
    // Child process: redirect output, execute
    freopen("/dev/null", "w", stdout);
    freopen("/dev/null", "w", stderr);
    execl(op_path, op_path, "player", "up", NULL);
    _exit(127);  // execl failed
} else if (pid > 0) {
    // Parent process: wait for child
    int status;
    waitpid(pid, &status, 0);
}
free(op_path);
```

Why this matters: When you press Ctrl+C, the signal handler sets `g_shutdown = 1`. The main loop exits cleanly. But if a `system()` call spawned a child, that child becomes orphaned and keeps running forever! With `fork()/exec()/waitpid()`, every child is always reaped.

[SEC 安] The CPU-Safe Module Template

This is the most important pattern in TPMOS. Every module follows this structure:

```
#include <signal.h>
#include <sys/stat.h>
#include <sys/wait.h>

static volatile sig_atomic_t g_shutdown = 0;

// Signal handler for graceful shutdown
static void handle_sigint(int sig) {
    (void)sig;
    g_shutdown = 1;
}

// CPU-Safe command execution
static int run_command(const char* cmd) {
    pid_t pid = fork();
    if (pid == 0) {
        freopen("/dev/null", "w", stdout);
        freopen("/dev/null", "w", stderr);
        execl("/bin/sh", "/bin/sh", "-c", cmd, (char *)NULL);
        _exit(127);
    } else if (pid > 0) {
        int status;
        waitpid(pid, &status, 0);
        return WIFEXITED(status) ? WEXITSTATUS(status) : -1;
    }
    return -1;
}

int main(void) {
    // STEP 1: Register signal handlers
    signal(SIGINT, handle_sigint);
    signal(SIGTERM, handle_sigint);

    // STEP 2: Set process group for clean cleanup
    setpgid(0, 0);

    // Initialize
    resolve_paths();
    update_state();
    trigger_render();

    // STEP 3: History file polling setup
    char *hist_path = build_path_malloc("pieces/apps/player_app/history.txt");
    long last_pos = 0;
    struct stat st;

    // STEP 4: Main loop with shutdown flag check
    while (!g_shutdown) {
        // STEP 5: Focus-aware throttling
        if (!is_active_layout()) {
            usleep(100000);  // 100ms when not in focus
```

```
continue;
```

```
}
```

```
// STEP 6: stat()-first polling
```

```
if (stat(hist_path, &st) == 0) {
```

```
if (st.st_size > last_pos) {
```

```
// New input available
```

```
FILE *hf = fopen(hist_path, "r");
```

```
if (hf) {
```

```
fseek(hf, last_pos, SEEK_SET);
```

```
int key;
```

```
while (fscanf(hf, "%d", &key) == 1) {
```

```
process_key(key);
```

```
update_state();
```

```
trigger_render();
```

```
}
```

```
last_pos = ftell(hf);
```

```
fclose(hf);
```

```
}
```

```
}
```

```
}
```

```
// STEP 7: Bounded sleep (60 FPS max)
```

```
usleep(16667);
```

```
}
```

```
// Cleanup
```

```
free(hist_path);
```

```
return 0;
```

```
}
```

Key Patterns:

1. Signal handling: `handle_sigint` sets flag, main loop checks it
2. Process group: `setpgid(0, 0)` ensures Ctrl+C kills all children
3. Focus throttling: 100ms sleep when not active, 16ms when active
4. stat()-first: Only read history if file size changed
5. Bounded sleep: `usleep(16667)` = 60 FPS max, never tight loop

Organizing Ops: The Centralized Directory Pattern

DO: Centralized Ops Directory

All ops for a project should live in ONE central location:

```
projects/<project_id>/ops/
├── +x/                    # Compiled binaries
│   ├── create_profile.+x
│   ├── auth_user.+x
│   └── get_session.+x
├── src/                   # Source code (optional)
│   ├── create_profile.c
│   ├── auth_user.c
│   └── get_session.c
└── ops_manifest.txt       # Registry of all ops (REQUIRED for Fondu)
```

ops_manifest.txt Format

```
# Comment lines start with #
# Format: op_name|description|args_format

create_profile|Creates a new user profile|project_path:string,username:string
auth_user|Authenticates a user session|project_path:string,username:string
get_session|Get current session data|project_path:string,username:string
```

project.pdl Registration

In your `project.pdl`, declare that your project exposes ops:

```
[META]
project_id = user
exposes_ops = true
entry_layout = projects/user/layouts/user.chtpm

[OPS]
ops_dir = projects/user/ops/
manifest = projects/user/ops/ops_manifest.txt
```

DON'T: Scatter Ops

```
projects/<project>/traits/move_entity.c     (scattered in traits/)
projects/<project>/plugins/move_entity.c    (scattered in plugins/)
projects/<project>/manager/move_entity.c    (scattered in manager/)
```

Fondu Installation of Ops

When you run `./fondu --install <project>`, Fondu will:

1. Read `project.pdl` → finds `exposes_ops = true`
2. Read `ops/ops_manifest.txt` → gets list of ops
3. Copy `ops/+x/.+x` to `pieces/apps/installed/<project>/ops/+x/`
4. Register ops in `pieces/os/ops_registry/<project>.txt`
5. Update `pieces/os/ops_catalog.txt`

Now other projects can call your ops via PAL scripts:

```
OP user::create_profile "player1"
OP user::auth_user "player1"
```

Code Example: Fondu Installer Flow

The Fondu installer (`pieces/system/fondu/fondu.c`) reads project metadata and copies files:

```c
// Simplified fondu.c install flow
void install_project(const char* project_id) {
    char pdl_path[4096], manifest_path[4096];
    snprintf(pdl_path, sizeof(pdl_path), "projects/%s/project.pdl", project_id);

    // 1. Read project.pdl to check exposes_ops
    if (!pdl_has_key(pdl_path, "exposes_ops")) return;

    // 2. Read ops_manifest.txt
    snprintf(manifest_path, sizeof(manifest_path),
             "projects/%s/ops/ops_manifest.txt", project_id);
    FILE *mf = fopen(manifest_path, "r");
    if (!mf) return;

    // 3. Copy each op to installed directory
    char line[1024];
    while (fgets(line, sizeof(line), mf)) {
        if (line[0] == '#') continue;  // Skip comments
        char *op_name = strtok(line, "|");
        char src[4096], dst[4096];
        snprintf(src, sizeof(src), "projects/%s/ops/+x/%s.+x", project_id, op_name);
        snprintf(dst, sizeof(dst), "pieces/apps/installed/%s/ops/+x/%s.+x", project_id, op_name);
        copy_file(src, dst);
    }
    fclose(mf);

    // 4. Register in ops_registry
    // 5. Update ops_catalog
}
```

Fondu Uninstall/Update

- Uninstall: `./fondu --uninstall <project>` removes files from `pieces/apps/installed/` and updates registry
- Update: `./fondu --update <project>` compares versions, copies only changed ops
- Dependency Resolution: If project A depends on project B's ops, Fondu installs B first

---

Code Example: The Canonical Op (`move_entity.c`)

The `move_entity` op is the most-called Muscle in TPMOS. Here's how it works:

```c
/* move_entity.c - The canonical TPMOS Op */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_PATH 8192

char project_root[MAX_PATH] = ".";
char current_project[256] = "template";

// Resolve paths from location_kvp
void resolve_paths() {
    FILE *kvp = fopen("pieces/locations/location_kvp", "r");
    if (kvp) {
        char line[1024];
        while (fgets(line, sizeof(line), kvp)) {
            char *eq = strchr(line, '=');
            if (eq) {
                *eq = '\0';
                if (strcmp(trim_str(line), "project_root") == 0)
                    snprintf(project_root, sizeof(project_root), "%s", trim_str(eq + 1));
            }
        }
        fclose(kvp);
    }
}

// Read integer state value
int get_state_int(const char* piece_id, const char* key) {
    char path[MAX_PATH];
    snprintf(path, sizeof(path), "%s/projects/%s/pieces/%s/state.txt",
             project_root, current_project, piece_id);
    FILE *f = fopen(path, "r");
    if (!f) return -1;
    char line[256]; int val = -1;
    while (fgets(line, sizeof(line), f)) {
        char *eq = strchr(line, '=');
        if (eq && strncmp(line, key, strlen(key)) == 0) {
            val = atoi(trim_str(eq + 1));
            break;
        }
    }
    fclose(f);
    return val;
}

// Write integer state value
void set_state_int(const char* piece_id, const char* key, int val) {
    char path[MAX_PATH];
    snprintf(path, sizeof(path), "%s/projects/%s/pieces/%s/state.txt",
```

```
project_root, current_project, piece_id);
```

```
// Read existing lines, update matching key, write back
```

```
// (Standard read-modify-write pattern - see CH2 for full code)
```

}

```
// Hit frame marker to trigger render
```

```
void hit_frame_marker() {
```

```
FILE *f = fopen("pieces/display/frame_changed.txt", "a");
```

```
if (f) { fprintf(f, "X\n"); fclose(f); }
```

}

```
int main(int argc, char *argv[]) {
```

```
if (argc < 3) return 1;
```

```
resolve_paths();
```

```
const char *id = argv[1];
```

```
const char *dir = argv[2];
```

```
int x = get_state_int(id, "pos_x");
```

```
int y = get_state_int(id, "pos_y");
```

```
int nx = x, ny = y;
```

```
// Calculate new position
```

```
if (strcmp(dir, "up") == 0) ny--;
```

```
else if (strcmp(dir, "down") == 0) ny++;
```

```
else if (strcmp(dir, "left") == 0) nx--;
```

```
else if (strcmp(dir, "right") == 0) nx++;
```

```
// Bounds check
```

```
if (nx < 0 || nx >= 20 || ny < 0 || ny >= 10) return 0;
```

```
// Collision check (read map tile at new position)
```

```
char tile = get_tile_at(nx, ny);
```

```
if (tile == '#' || tile == 'R') return 0;  // Blocked
```

```
// Treasure tile interaction
```

```
if (tile == 'T') {
```

```
int hap = get_state_int(id, "happiness");
```

```
if (hap != -1) set_state_int(id, "happiness", (hap < 90) ? hap + 10 : 100);
```

```
}
```

```
// Update position
```

```
set_state_int(id, "pos_x", nx);
```

```
set_state_int(id, "pos_y", ny);
```

```
// Energy cost
```

```
int enr = get_state_int(id, "energy");
```

```
if (enr != -1) set_state_int(id, "energy", (enr > 0) ? enr - 1 : 0);
```

```
// Trigger render
```

```
hit_frame_marker();
```

```
return 0;
```

}

What this Op demonstrates:

1. Path resolution: Always reads from `location_kvp` first
2. State sovereignty: Reads/writes plaintext `state.txt` files
3. Single responsibility: Does ONE thing (move entity) and exits
4. Collision detection: Checks map tiles before moving
5. Tile interaction: Special behavior on treasure tiles
6. Render trigger: Hits `frame_changed.txt` to trigger pipeline
7. No `system()` calls: Pure file I/O, no subprocess spawning

External Ops KVP (Advanced)

For advanced setups, you can maintain an external KVP file:

File: `projects/<project_id>/ops/ops_registry.kvp`

Format:

```
op_create_profile=projects/user/ops/create_profile.c|username:string
op_auth_user=projects/user/ops/auth_user.c|username:string
op_get_session=projects/user/ops/get_session.c|username:string
```

Use Cases:

- Dynamic op discovery
- Ops that span multiple directories
- External/trunk ops that aren't in `projects/`

Why Centralize Ops?

Benefit	Explanation
Fondu Bundling	Fondu can install all ops as a single bundle

Discovery	Other projects discover ops via `--list-ops`
Separation	Clean separation of "muscle" (ops) from "brain" (manager)
Testing	Easy to test ops standalone (no manager needed)
Reusability	Reusable across projects (PAL scripts call by `project::op_name`)
Single Truth	Single source of truth for what ops exist

[TOOL 工] The Developer Workflow

1. Pick a Template: Copy `#.docs/future-facing/cpu_safe_module_template.c`.
2. Define your Logic: What happens when the user presses 'W'? (Call `move_entity.+x north`).
3. Update the Mirror: Write the new coordinates to `state.txt`.
4. Pulse the Frame: Tell the Renderer that something changed! [PULSE 心]

[TIPS 明] **Pro Tip:** Keep your Muscles "dumb" and your Brains "thin". If a Muscle can do it, don't put it in the Brain! [FAST 快]

[WARN 警] Common Pitfalls

Pitfall 1: Orphaned Processes on Ctrl+C

Symptom: After quitting, the game keeps running in the background (music plays, files change).
Cause: Missing signal handler or using `system()` instead of `fork()/exec()`.
Fix: Always register `handle_sigint` and use `setpgid(0, 0)`:

```
signal(SIGINT, handle_sigint);
signal(SIGTERM, handle_sigint);
setpgid(0, 0);
```

Pitfall 2: Tight Loop Without Sleep

Symptom: TPMOS process uses 100% CPU even when idle.

Cause: Main loop has no `usleep()` or uses `usleep(0)`.

Fix: Always sleep at the end of each iteration:

```
// When active: 60 FPS
usleep(16667);
// When inactive: 10 FPS
usleep(100000);
```

Pitfall 3: Scattering Ops in Wrong Directories

Symptom: Fondu can't find your ops, other projects can't call them.

Cause: Ops placed in `traits/`, `plugins/`, or `manager/` instead of centralized `ops/`.

Fix: All ops go in `projects/<id>/ops/` with `ops_manifest.txt`.

Pitfall 4: Forgetting `ops_manifest.txt`

Symptom: `fondu --install` skips your project.

Cause: No manifest file or wrong format.

Fix: Create `ops/ops_manifest.txt` with `op_name|description|args_format` format.

Pitfall 5: Thick Brain Anti-Pattern

Symptom: Manager is 5000+ lines, crashes take down everything, hard to debug.

Cause: Putting movement, hunger, rendering, AI all in one manager loop.

Fix: Move logic into separate ops. Manager should only route input and call ops.

[BLD 馆] Scholar's Corner: The "Thin Brain" Revolution

In the early days of TPMOS, the `fuzz_legacy_manager.c` was over 5,000 lines of code. It handled movement, hunger, the market, and rendering all in one massive loop. It was a "Thick Brain." When one part of the code crashed, the entire pet simulation died! The "Thin Brain" revolution happened when we moved the movement logic into `move_entity.+x` and the hunger logic into PAL scripts. Suddenly, the manager was under 500 lines. The system became stable, and developers could finally sleep at night. [IDLE 眠][ZEN 禅]

Modern Example: The User Project (April 2026)

The `projects/user/` project is the canonical example of centralized ops:

```
projects/user/
├── project.pdl (exposes_ops = true)
├── ops/
│   ├── +x/create_profile.+x
│   ├── +x/auth_user.+x
│   ├── +x/get_session.+x
│   └── ops_manifest.txt
└── manager/+x/user_manager.+x
```

When installed via Fondu (`./fondu --install user`), all three ops become available to every other project via PAL scripts. This is the "Ops as a Service" pattern that powers TPMOS. [FAST 快]

[DOC 记] Study Questions

1. Explain the difference between a "Module" and an "Op."
2. Why is the `fork()` / `exec()` pattern preferred over the `system()` command?
3. What are the three core rules of the "CPU-Safe Mandate"?
4. True or False: A Module should sleep longer when its app is not the "active layout."
5. Short Answer: What is the correct directory structure for organizing ops in a project?
6. Scenario: You want to create a new op called `teleport_entity`. Where do you put it, and what file do you update to make it discoverable by Fondu?
7. Critical Thinking: Why is centralizing ops better than scattering them in `traits/`, `plugins/`, etc.?

[Return to Index](INDEX.md)

[APP 厂] Chapter 5: The App Factory

How do you build a new tool for TPMOS? It's easier than you think! We call this the App Factory pattern. [TOOL 工][BLD 建]

[DNA 命] What is an App?

In TPMOS, an "App" is just a directory with a specific DNA file. [DNA 命]

The App Anatomy

```
pieces/apps/my_new_app/
├── my_new_app.pdl      # The App DNA
├── layouts/            # The UI screens (.chtpm files)
├── manager/            # The Brain (+x/module.+x)
└── pieces/             # Local pieces used by the app
```

[DOC 记] The App DNA (`.pdl`)

Every app needs a descriptor so the OS knows how to launch it.

```
<app_id>op-ed</app_id>
<version>6.0</version>
<layout_path>pieces/apps/op-ed/layouts/main.chtpm</layout_path>
<module_path>pieces/apps/op-ed/manager/+x/op-ed_module.+x</module_path>
```

[TOOL 工] Featured Apps

TPMOS comes with several powerful system tools:

`op-ed` (The Piece Editor)

The most important tool! It allows you to create, delete, and modify Pieces in real-time.

- **Feature:** Roundtrip Parity. You can edit a file, save it, and the game updates instantly.

[PET 爪] `fuzz-ops` (The Pet Manager)

A complex simulation app that manages "Fuzzpets".

- **Feature:** It uses **PAL (Prisc Assembly Language)** to give pets AI behaviors like "Hunger" and "Play".

[USER 人] `user` (The Identity Manager)

Handles logins, profiles, and preferences.

- **Feature:** Persistent user state across different projects.

[EXP 卫] The Horizon: Upcoming Apps

Several high-impact apps are currently in the incubation phase:

- [BOT 机] **AI-Labs**: A laboratory for local LLM (like Qwen/Gemma) integration. Used for tech R&D and autonomous codegen. (Covered in CH10)
- [LUNA 阳] **LSR (Lunar Streetrace Raider)**: A "Civ-Lite" simulation where AI agents build lunar civilizations. (Covered in CH10)
- [NET 链] **P2P-NET**: A decentralized networking layer with blockchain and NFT trading. (Covered in CH10)

For deep-dives into fuzz-op (pet simulation) and op-ed (piece editor), see CH7. For PAL scripting used by these apps, see CH6.

Code Example: App Manager Loop

Every app's manager follows the same CPU-safe pattern (see CH4 for the full template). Here's a simplified example from `player_manager.c`:

```c
// player_manager.c - Main app loop
int main(void) {
    signal(SIGINT, handle_sigint);
    setpgid(0, 0);
    resolve_paths();

    char *hist_path = build_path_malloc("pieces/apps/player_app/history.txt");
    long last_pos = 0;
    struct stat st;

    while (!g_shutdown) {
        if (!is_active_layout()) {
            usleep(100000);  // 100ms when not in focus
            continue;
        }

        if (stat(hist_path, &st) == 0 && st.st_size > last_pos) {
            FILE *hf = fopen(hist_path, "r");
            if (hf) {
                fseek(hf, last_pos, SEEK_SET);
                int key;
                while (fscanf(hf, "%d", &key) == 1) {
                    route_input(key);  // Route to appropriate op
                }
                last_pos = ftell(hf);
                fclose(hf);
            }
        }
        usleep(16667);  // 60 FPS
    }
    free(hist_path);
    return 0;
}
```

The `route_input()` function decides which op to call based on the key pressed.

Code Example: op-ed Module (Editor App)

The `op-ed` module handles cursor movement and tile placement:

```
// op-ed_module.c - Editor input handling
void process_key(int key) {
    int cx = get_state_int("editor", "cursor_x");
    int cy = get_state_int("editor", "cursor_y");

    if (key == 'w' || key == 1002) cy--;       // Up
    else if (key == 's' || key == 1003) cy++;  // Down
    else if (key == 'a' || key == 1000) cx--;  // Left
    else if (key == 'd' || key == 1001) cx++;  // Right
    else if (key == 10 || key == 13) {
        // ENTER: Place selected tile
        place_tile_at(cx, cy, current_tile);
    }
    else if (key == 27) {
        // ESC: Return to selector
        strcpy(active_target_id, "selector");
    }

    set_state_int("editor", "cursor_x", cx);
    set_state_int("editor", "cursor_y", cy);
    hit_frame_marker();  // Trigger render
}
```

[DOC 记] App DNA with Methods

An app's `.pdl` file can declare methods (actions the app can perform):

```
META | app_id | fuzz-op
META | version | 2.0
META | layout_path | projects/fuzz-op/layouts/fuzz-op.chtpm
META | module_path | projects/fuzz-op/manager/+x/fuzz-op_manager.+x

METHOD | feed | pieces/apps/playrm/ops/+x/fuzzpet_action.+x fuzzpet feed
METHOD | sleep | pieces/apps/playrm/ops/+x/fuzzpet_action.+x fuzzpet sleep
METHOD | play | pieces/apps/playrm/ops/+x/fuzzpet_action.+x fuzzpet play
```

Methods are callable by PAL scripts and the UI. See CH6 for PAL integration.

Fondu App Installation

Fondu doesn't just install ops - it can install entire apps:

```
./fondu --install user
```

This copies:

1. `manager/+x/user_manager.+x` → `pieces/apps/installed/user/manager/`
2. `ops/+x/.+x` → `pieces/apps/installed/user/ops/+x/`
3. `layouts/.chtpm` → `pieces/apps/installed/user/layouts/`
4. Updates `os.chtpm` button registry

[TOOL 工] Developer Example: Creating a New App

Step 1: Directory Structure

```
pieces/apps/my_app/
├── my_app.pdl
├── layouts/
│   └── main.chtpm
├── manager/
│   └── my_app.c
└── pieces/
    └── my_piece/
        └── state.txt
```

Step 2: Minimum Viable Manager (~50 lines)

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/stat.h>

static volatile sig_atomic_t g_shutdown = 0;
static void handle_sigint(int sig) { g_shutdown = 1; }

int main(void) {
    signal(SIGINT, handle_sigint);
    setpgid(0, 0);

    // Initialize state
    FILE *f = fopen("pieces/apps/my_app/pieces/my_piece/state.txt", "w");
    if (f) {
        fprintf(f, "name=MyApp\nstatus=running\n");
        fclose(f);
    }

    // Simple loop
    while (!g_shutdown) {
        // Check for input, update state, trigger render
        usleep(16667);  // 60 FPS
    }
    return 0;
}
```

Step 3: Add a Layout (`main.chtpm`)

```
<layout id="my_app.chtpm">
    <row>
        <cell>My App</cell>
    </row>
    <row>
        <cell>Status: ${status}</cell>
    </row>
</layout>
```

Step 4: Register in `os.chtpm`

Add a button to `pieces/chtpm/layouts/os.chtpm`:

```
<button label="My App" onClick="KEY:a" href="pieces/apps/my_app/my_app.pdl" />
```

Step 5: Compile and Test

```
gcc -o pieces/apps/my_app/manager/+x/my_app.+x pieces/apps/my_app/manager/my_app.c -pthread
./button.sh run
```

Press `a` to launch your app!

[WARN 警] Common Pitfalls

Pitfall 1: Forgetting `is_active_layout()` Check

Symptom: App consumes CPU even when not on screen.

Fix: Always check layout before processing:

```
if (!is_active_layout()) { usleep(100000); continue; }
```

Pitfall 2: Not Registering App in `os.chtpm`

Symptom: App compiles but can't be launched from main menu.

Fix: Add `<button>` to `os.chtpm` with correct `href` to `.pdl` file.

Pitfall 3: App DNA Path Mismatch

Symptom: OS can't find the module binary.

Fix: Ensure `module_path` in `.pdl` matches actual binary location (`+x/` directory).

[LAUNCH 发] Launching your App

To see your app in the OS, you just need to add a button to the main `os.chtpm` layout:

```
<button label="[LAUNCH 发] My New App" onClick="KEY:m" href="pieces/apps/my_new_app/my_new_app.pdl...
```

[TIPS 明] **Pro Tip:** Start by duplicating an existing app (like `playrm`) and changing the

names. It's the fastest way to learn! [SPRINT 跑][FAST 速]

[BLD 馆] Scholar's Corner: The "Day the Editor Became the Game"

One of the most profound moments in TPMOS history was when a developer accidentally launched the `op-ed` (the editor) inside the `fuzz-op` (the pet game). Instead of crashing, the system simply rendered the editor's selector on top of the pet's map. The developer realized they could "play" the game by editing the world in real-time—placing walls to trap pets or changing their health with a few keystrokes. This blurred the line between playing and building, leading to our philosophy that "The Editor is the Game."

[DOC 记] Study Questions

1. What is the minimum requirement for a directory to be considered an "App" in TPMOS?
2. Which app is responsible for managing user profiles?
3. How do you add a new app to the main OS menu?
4. True or False: Every app must have its own `.pdl` file.

[Return to Index](INDEX.md)

Chapter 6: PAL: The Assembly Language of TPMOS

How do you orchestrate complex operations across an entire project? You write a PAL script. PAL (Prisc Assembly Language) is the glue that holds TPMOS together. [LINK 接][MIND 脑]

What is PAL?

PAL is an assembly-style scripting language designed specifically for TPMOS. It lets you chain together Ops (Muscles) into complex workflows without writing C code.

The Instruction Set

PAL has a RISC-inspired instruction set:

Instruction	Purpose	Example
`OP`	Call an external op	`OP playrm::move_entity "player" "up"`
`call`	Call a subroutine (label)	`call game_loop`
`sleep`	Pause execution (ms)	`sleep 100`
`beq`	Branch if equal	`beq r0, r1, done`
`jalr`	Jump and link register	`jalr handler`
`j`	Unconditional jump	`j start`
`lw`	Load word from memory	`lw r1, 0(r2)`
`sw`	Store word to memory	`sw r1, 0(r2)`
`addi`	Add immediate	`addi r1, r0, 1`
`halt`	Stop execution	`halt`
`read_history`	Read from history buffer	`read_history r1`
`read_state`	Read piece state	`read_state r1, "player", "pos_x"`
`hit_frame`	Trigger render	`hit_frame`
`read_pos`	Read entity position	`read_pos r1, r2, "player"`

Register Model

PAL uses 16 general-purpose registers (`r0`-`r15`) and a 4096-word memory space. Labels provide control flow targets.

Code Example: The PAL Interpreter

The PAL interpreter (`pieces/system/prisc/prisc+x.c`, 771 lines) parses and executes PAL scripts. Here's the core execution loop:

```c
// prisc+x.c - Core instruction dispatch
typedef struct {
    char name[32];
    char handler[256];  // Path to .+x binary
    char desc[128];
} CustomOp;

CustomOp custom_ops[MAX_OPS];
int op_count = 0;

void execute_instruction(int pc) {
    Inst *inst = &program[pc];

    switch (inst->op) {
        case OP_CUSTOM: {
            // Find and execute the custom op
            for (int i = 0; i < op_count; i++) {
                if (strcmp(custom_ops[i].name, inst->custom_name) == 0) {
                    // Execute via fork()/exec()
                    char cmd[4096];
                    snprintf(cmd, sizeof(cmd), "%s %s %s",
                             custom_ops[i].handler,
                             inst->literal_arg,
                             inst->literal_arg2);
                    run_command(cmd);
                    break;
                }
            }
            break;
        }
        case OP_SLEEP:
            usleep(inst->imm * 1000);  // Convert ms to us
            break;
        case OP_BEQ:
            if (regs[inst->rs1] == regs[inst->rs2])
                pc = find_label(inst->label_ref);
            break;
        case OP_HIT_FRAME:
            hit_frame_marker();  // Trigger render pipeline
            break;
        // ... more instructions
    }
}
```

Key insight: PAL scripts compile to an in-memory instruction array, then execute sequentially. Custom ops are dispatched via `fork()/exec()` - the same CPU-safe pattern as C modules.

[DOC 记] PAL Program Example

Here's a simple PAL script that creates a user, moves them, and checks for treasure:

```
; welcome_sequence.asm - New player welcome flow
start:
    ; Create user profile
    call user::create_profile "player1"
    sleep 200

    ; Move player to starting position
    OP playrm::move_entity "player1" "right"
    sleep 100
    OP playrm::move_entity "player1" "right"
    sleep 100

    ; Read player position
    read_pos r1, r2, "player1"
    addi r3, r0, 5        ; r3 = 5 (target X)
    beq r1, r3, found_treasure

    ; No treasure, continue normally
    OP playrm::render_map
    halt

found_treasure:
    ; Player reached treasure!
    OP playrm::fuzzpet_action "player1" "celebrate"
    hit_frame
    halt
```

Code Example: Custom Op Registration

Ops become PAL-callable through the `CustomOp` registry:

```
// Registering an op for PAL use
void register_custom_op(const char* name, const char* handler_path, const char* desc) {
    if (op_count >= MAX_OPS) return;

    strncpy(custom_ops[op_count].name, name, 31);
    strncpy(custom_ops[op_count].handler, handler_path, 255);
    strncpy(custom_ops[op_count].desc, desc, 127);
    op_count++;
}

// Usage (called during PAL initialization):
register_custom_op(
    "playrm::move_entity",
    "pieces/apps/playrm/ops/+x/move_entity.+x",
    "Move an entity in a direction"
);
```

Once registered, the op is callable from any PAL script via `OP playrm::move_entity "player" "up"`.

PAL + Fondu Integration

When you install a project via Fondu, PAL scripts are installed alongside ops:

```
./fondu --install user
```

This copies:

1. `ops/+x/.+x` → `pieces/apps/installed/user/ops/+x/`
2. `scripts/.asm` → `pieces/apps/installed/user/scripts/`
3. Updates `ops_catalog.txt` with available PAL-callable ops

Now any project can call:

```
OP user::create_profile "new_user"
OP user::auth_user "new_user"
```

Cross-Project PAL Calls

PAL supports calling ops from any installed project:

```
; This script calls ops from multiple projects
OP user::create_profile "player1"
OP playrm::move_entity "player1" "right"
OP p2p::connect_peer "node_01"
```

[DOC 单] PAL Best Practices

1. Keep scripts under 100 lines. Long scripts are hard to debug. Split into subroutines with `call`.
2. Use `call` for subroutines, `OP` for external ops. `call` jumps to a label within the script; `OP` forks a new process.
3. Always `sleep` between heavy operations. Give the filesystem time to flush:

```asm
OP playrm::render_map
sleep 50  ; Let render complete before next action
```

4. Error handling with `beq` checks:

```asm
read_state r1, "player", "health"
beq r1, r0, player_dead  ; If health == 0, jump
```

5. Always end with `halt`. Without it, the interpreter runs off the end of the program.

[BLD 馆] Architectural Note: PAL as the Glue

PAL sits at the center of the TPMOS architecture:

```
┌──────────────────────────┐
│   MODULE     │ The Brain (C code)
│  (Manager)   │
└──────────────┬───────────┘
       │ calls
       ▼
┌──────────────────────────┐
│    PAL       │ The Glue (Assembly scripts)
│  (prisc+x)   │
└──────────────┬───────────┘
       │ calls
       ▼
┌──────────────────────────┐
│     OP       │ The Muscle (C binaries)
│  (move_etc)  │
└──────────────┬───────────┘
       │ updates
       ▼
┌──────────────────────────┐
│  STATE.TXT   │ The Reality (Plaintext files)
│   (Disk)     │
└──────────────┬───────────┘
       │ triggers
       ▼
┌──────────────────────────┐
│  RENDERER    │ The Display (ASCII or GL)
│  (Screen)    │
└──────────────────────────┘
```

Modules (Brains) call PAL scripts, which call Ops (Muscles), which update Piece state files, which trigger Renders. This is the complete TPMOS pipeline.

Developer Exercise: Write a PAL Script

Write a PAL script that:

1. Creates a user profile
2. Moves the player 5 steps right
3. Checks if player reached a treasure tile
4. Updates happiness if treasure found
5. Renders the final state

Solution:

```
; treasure_hunt.asm
start:
    OP user::create_profile "hunter"
    sleep 200

    ; Move 5 steps right
    OP playrm::move_entity "hunter" "right"
    sleep 100
    OP playrm::move_entity "hunter" "right"
    sleep 100
    OP playrm::move_entity "hunter" "right"
    sleep 100
    OP playrm::move_entity "hunter" "right"
    sleep 100
    OP playrm::move_entity "hunter" "right"
    sleep 100

    ; Check happiness (treasure interaction is automatic in move_entity)
    read_state r1, "hunter", "happiness"
    addi r2, r0, 60
    beq r1, r2, treasure_found  ; If happiness >= 60, likely found treasure

    ; No treasure
    OP playrm::render_map
    halt

treasure_found:
    OP playrm::fuzzpet_action "hunter" "celebrate"
    hit_frame
    OP playrm::render_map
    halt
```

[WARN 警] Common Pitfalls

Pitfall 1: Infinite Loops in PAL Scripts

Symptom: System hangs, CPU spikes to 100%.

Cause: `j` instruction without proper exit condition, or recursive `call` without base case.

Fix: Always ensure loops have a terminating condition:

```
; [FAIL 错] BAD: Infinite loop
loop:
    OP playrm::move_entity "player" "right"
    j loop

; [OK 是] GOOD: Bounded loop
addi r1, r0, 0    ; counter = 0
addi r2, r0, 5    ; max = 5
loop:
    beq r1, r2, done
    OP playrm::move_entity "player" "right"
    addi r1, r1, 1
    sleep 100
    j loop
done:
    halt
```

Pitfall 2: Forgetting `sleep` Between Heavy Ops

Symptom: Ops fail silently, state doesn't update.

Cause: Filesystem hasn't flushed previous write before next op reads.

Fix: Add `sleep 50-200` between ops that read/write state:

```
OP playrm::move_entity "player" "right"
sleep 100  ; Let state flush to disk
OP playrm::render_map
```

Pitfall 3: Cross-Project Call Failures

Symptom: `OP other_project::some_op` does nothing.

Cause: Target project not installed via Fondu, or op not in catalog.

Fix: Run `./fondu --install other_project` first, verify op exists in `ops_catalog.txt`.

Pitfall 4: Not Ending with `halt`

Symptom: Interpreter crashes or executes garbage memory after script.

Cause: Script falls off the end without `halt`.

Fix: Every execution path must end with `halt`.

[BLD 馆] Scholar's Corner: The "Assembly That Assembled Itself"

In the early days of TPMOS, a developer wrote a PAL script to automate the process of creating new PAL scripts. The script would read a template, substitute variable names, and write the output to a new `.asm` file. One day, the developer accidentally pointed the script at itself. The meta-script generated a new version of itself, which generated another, which generated another... Within seconds, there were 10,000 copies of the script filling the disk. This became known as "The Assembly That Assembled Itself." It taught us the critical importance of output path validation and the power (and danger) of self-referential systems. [INF 无]

[DOC 记] Study Questions

1. What is the difference between `call` and `OP` in PAL?
2. Why does PAL use `fork()/exec()` for custom ops instead of direct function calls?
3. Explain the role of `sleep` in PAL scripts. What happens if you remove it?
4. Write a PAL script that moves a player from position (0,0) to (3,3) diagonally (right, down, right, down, right, down).
5. True or False: PAL scripts share memory with the module that calls them.
6. Scenario: You wrote a PAL script that calls `OP user::create_profile`, but nothing happens. What are three possible causes?
7. Critical Thinking: Why is PAL designed as an assembly language rather than a high-level scripting language like Python?

[Return to Index](INDEX.md)

Chapter 7: fuzz-op & op-ed: The Flagship Apps

TPMOS comes with two flagship applications that demonstrate the full power of the Piece architecture: fuzz-op (a pet simulation) and op-ed (a real-time piece editor). Together, they prove that "The Editor is the Game."

[PET 爪] fuzz-op: The Pet Simulation

Architecture Overview

fuzz-op is a pet simulation project that demonstrates the complete TPMOS pipeline:

- Manager: `projects/fuzz-op/manager/fuzz-op_manager.c` (~600 lines)
- Zombie AI: `pieces/apps/fuzzpet_app/traits/zombie_ai.c` (~200 lines)
- Ops: Uses `move_entity.+x`, `render_map.+x`, and custom fuzzpet actions
- Pieces: `fuzzpet`, `zombie_01`, `selector`, and map tiles

Code Example: fuzz-op Manager Main Loop

The manager handles input routing, selector system, Z-level changes, and zombie AI calls:

```c
// fuzz-op_manager.c - Input routing
void route_input(int key) {
    // Handle quit key (return to selector)
    if (key == '9' || key == 27) {
        strcpy(active_target_id, "selector");
        log_resp("Returned to Selector.");
    }

    // Handle selection (ENTER selects entity at cursor)
    if (strcmp(active_target_id, "selector") == 0 && key == 10) {
        int cx = get_state_int_fast("selector", "pos_x");
        int cy = get_state_int_fast("selector", "pos_y");

        // Find entity at selector position
        DIR *dir = opendir(pieces_dir);
        struct dirent *entry;
        while ((entry = readdir(dir)) != NULL) {
            if (entry->d_name[0] == '.') continue;
            if (strstr(entry->d_name, "zombie") != NULL) continue;  // Can't select zombies
            int tx = get_state_int_fast(entry->d_name, "pos_x");
            int ty = get_state_int_fast(entry->d_name, "pos_y");
            if (tx == cx && ty == cy) {
                strncpy(active_target_id, entry->d_name, 63);
                log_resp("Selected %s.", entry->d_name);
                break;
            }
        }
        closedir(dir);
    }

    // Handle movement (WASD or arrows)
    if (key == 'w' || key == 'a' || key == 's' || key == 'd' ||
        (key >= 1000 && key <= 1003)) {
        char* trait = build_path_malloc("pieces/apps/playrm/ops/+x/move_entity.+x");
        char dir_str[16];
        if (key == 'w' || key == 1002) strcpy(dir_str, "up");
        else if (key == 's' || key == 1003) strcpy(dir_str, "down");
        else if (key == 'a' || key == 1000) strcpy(dir_str, "left");
        else if (key == 'd' || key == 1001) strcpy(dir_str, "right");

        pid_t p = fork();
        if (p == 0) {
            freopen("/dev/null", "w", stdout);
            execl(trait, trait, active_target_id, dir_str, NULL);
            exit(1);
        } else waitpid(p, NULL, 0);
        free(trait);

        // Call zombie AI to chase player
        char* zombie_cmd = build_path_malloc(
            "pieces/apps/fuzzpet_app/traits/+x/zombie_ai.+x");
```

```
p = fork();
```

```
if (p == 0) {
```

```
freopen("/dev/nu11", "w", stdout);
```

```
execl(zombie_cmd, zombie_cmd, "zombie_01", active_target_id, NULL);
```

```
exit(1);
```

```
} else waitpid(p, NULL, 0);
```

```
free(zombie_cmd);
```

```
}
```

```
perform_mirror_sync();
```

```
save_manager_state();
```

```
trigger_render();
```

}

Key patterns demonstrated:

1. Selector system: Press 9 to return to selector, ENTER to select entity at cursor
2. Z-level changes: `x` = up, `z` = down (range 0-2 for sky/ground)
3. Zombie AI calls: After every player move, zombie_ai.+x is called to chase the target
4. Mirror sync: Updates fuzzpet_mirror state for UI display
5. Render trigger: Calls render_map after every state change

Code Example: Zombie AI Chase Logic

The zombie AI (`zombie_ai.c`) implements simple chase behavior:

```c
// zombie_ai.c - Chase AI implementation
int main(int argc, char *argv[]) {
    if (argc < 3) return 1;

    const char *zombie_id = argv[1];
    const char *target_id = argv[2];

    resolve_paths();  // From location_kvp

    // Get zombie position
    int zx = get_state_int(zombie_id, "pos_x");
    int zy = get_state_int(zombie_id, "pos_y");
    int zz = get_state_int(zombie_id, "pos_z");
    if (zx == -1 || zy == -1) return 0;  // Zombie doesn't exist
    if (zz == -1) zz = 0;

    // Get target (player) position
    int tx = get_state_int(target_id, "pos_x");
    int ty = get_state_int(target_id, "pos_y");
    int tz = get_state_int(target_id, "pos_z");
    if (tz == -1) tz = 0;

    // Only chase if on same Z level
    if (zz != tz) return 0;

    // Don't chase if already adjacent (attack range)
    int dist = abs(tx - zx) + abs(ty - zy);
    if (dist <= 1) return 0;

    // Simple chase: move X first, then Y
    int next_x = zx, next_y = zy;
    if (tx > zx) next_x++;
    else if (tx < zx) next_x--;
    else if (ty > zy) next_y++;
    else if (ty < zy) next_y--;

    // Bounds check (0-19 map)
    if (next_x < 0) next_x = 0;
    if (next_x > 19) next_x = 19;
    if (next_y < 0) next_y = 0;
    if (next_y > 19) next_y = 19;

    // Update zombie position
    set_state_int(zombie_id, "pos_x", next_x);
    set_state_int(zombie_id, "pos_y", next_y);

    return 0;
}
```

Piece State Examples

```
# zombie_01/state.txt
name=Zombie
type=zombie
pos_x=9
pos_y=2
pos_z=0
on_map=1
behavior=aggressive
speed=1
map_id=map_01_z0.txt

# selector/state.txt
name=Selector
type=selector
pos_x=5
pos_y=4
pos_z=0
on_map=1
map_id=map_01_z0.txt
```

Future Features

- Multi-zombie support: AI flocking behavior, multiple zombies chasing
- Hunger/happiness decay: Stats decrease over time, requiring player attention
- Breeding/genetics system: Pets reproduce with inherited traits
- Inventory and item collection: Pick up and use items
- Multi-map exploration: Z-levels 0-2 with different maps
- GL-OS 3D rendering: Pet world rendered in full 3D (see CH8)

---

op-ed: The Piece Editor

Architecture Overview

op-ed is a real-time grid editor for creating and modifying TPMOS maps:

- Manager: `projects/op-ed/manager/op-ed_manager.c`
- Features: Cursor movement, tile placement, MAPS and GLYPHS palettes, save/load
- Roundtrip Parity: Edit a file, save it, and the game updates instantly

Code Example: op-ed Module

The editor handles cursor movement and tile placement:

```
// op-ed_module.c - Editor input handling
void process_key(int key) {
    int cx = get_state_int("editor", "cursor_x");
    int cy = get_state_int("editor", "cursor_y");

    // Cursor movement
    if (key == 'w' || key == 1002) cy--;       // Up
    else if (key == 's' || key == 1003) cy++;  // Down
    else if (key == 'a' || key == 1000) cx--;  // Left
    else if (key == 'd' || key == 1001) cx++;  // Right

    // Place tile (ENTER)
    else if (key == 10 || key == 13) {
        place_tile_at(cx, cy, current_tile);
        hit_frame_marker();  // Instant update!
    }

    // Switch palette (2 = MAPS, 3 = GLYPHS)
    else if (key == '2') current_palette = PALETTE_MAPS;
    else if (key == '3') current_palette = PALETTE_GLYPHS;

    // Toggle emoji rendering (4)
    else if (key == '4') emoji_mode = !emoji_mode;

    // Quit (ESC)
    else if (key == 27) {
        strcpy(active_target_id, "selector");
    }

    set_state_int("editor", "cursor_x", cx);
    set_state_int("editor", "cursor_y", cy);
    hit_frame_marker();
}
```

Map File Format

Maps are simple text grids - what you see is what you get:

```
####################
#  ........@......T#
#  ......Z........T#
#  ...XR...@......T#
#  ....R..........T#
#  ....R......@...T#
#  ....R..........T#
#  ................#
#                  #
####################
```

Tile Legend:

- `#` = Wall
- `.` = Empty floor
- `@` = Decorative item
- `T` = Treasure
- `R` = Resource
- `X` = Player start
- `Z` = Zombie spawn
- ` ` (space) = Void (outside map)

Palette System

op-ed has two palettes:

- MAPS: Wall, floor, treasure, resources, player start, zombie spawn
- GLYPHS: Decorative characters (@, , +, etc.) for detail work

Press `2` for MAPS, `3` for GLYPHS. Press `4` to toggle emoji rendering.

Roundtrip Parity

The key feature of op-ed is roundtrip parity: when you place a tile and press ENTER, the change is written directly to the map file. The next render cycle reads the updated file and displays it instantly. There's no "save" button - the file IS the game state.

Future Features

- Multi-layer editing: Edit Z-levels 0-2 simultaneously

- Bulk operations: Fill regions, copy/paste map sections
- Template library: Pre-built map sections (rooms, corridors, dungeons)
- GL-OS live preview: Edit in 2D, see in 3D simultaneously
- Collaborative editing: Multi-user via P2P-NET (see CH10)
- Undo/redo history: Full edit history with rollback

[LINK 接] Shared Architecture Patterns

Both fuzz-op and op-ed demonstrate core TPMOS principles:

Pattern	fuzz-op	op-ed
12-step pipeline	[OK 是] Full pipeline	[OK 是] Full pipeline
Read/write Piece state	[OK 是] zombie_01, selector	[OK 是] editor cursor
fork()/exec() for ops	[OK 是] move_entity, zombie_ai	[OK 是] place_tile
Trigger renders via frame_changed.txt	[OK 是]	[OK 是]
CPU-safe mandates	[OK 是] Signal handling, throttling	[OK 是] Signal handling, throttling
PAL script integration	[OK 是] fuzzpet actions	[OK 是] (future)

[TOOL 工] Developer Example: Extending fuzz-op

Let's add a new feature: pet food that the player can place and the fuzzpet can eat.

Step 1: Add New Piece Type

Create `projects/fuzz-op/pieces/pet_food/state.txt`:

```
name=Pet Food
type=item
pos_x=0
pos_y=0
pos_z=0
on_map=0
nutrition=20
```

Step 2: Write the `eat_food` Op

Create `projects/fuzz-op/ops/src/eat_food.c`:

```
// eat_food.c - Pet eats food, gains energy
int main(int argc, char *argv[]) {
    if (argc < 3) return 1;
    const char *pet_id = argv[1];
    const char *food_id = argv[2];

    resolve_paths();

    int nutrition = get_state_int(food_id, "nutrition");
    if (nutrition <= 0) return 0;  // No nutrition left

    int energy = get_state_int(pet_id, "energy");
    if (energy != -1) {
        int new_energy = energy + nutrition;
        if (new_energy > 100) new_energy = 100;
        set_state_int(pet_id, "energy", new_energy);
    }

    // Remove food (set on_map=0)
    set_state_int(food_id, "on_map", 0);

    hit_frame_marker();
    return 0;
}
```

Compile: `gcc -o projects/fuzz-op/ops/+x/eat_food.+x projects/fuzz-op/ops/src/eat_food.c`

Step 3: Update Manager

In `fuzz-op_manager.c`, add handling for the 'e' key:

```
if (key == 'e') {
    // Find nearest food and eat it
    char* cmd = build_path_malloc(
        "projects/fuzz-op/ops/+x/eat_food.+x fuzzpet pet_food");
    run_command(cmd);
    free(cmd);
}
```

Step 4: Test with op-ed

1. Launch op-ed (`button.sh run`, select op-ed from menu)
2. Place a `pet_food` piece on the map
3. Switch back to fuzz-op
4. Move fuzzpet adjacent to food
5. Press 'e' - fuzzpet should gain energy!

[WARN 警] Common Pitfalls

Pitfall 1: Zombie AI Not Moving (Hardcoded Paths)

Symptom: Zombie stays in place, never chases player.
Cause: `zombie_ai.+x` binary doesn't exist, or source has hardcoded paths like `projects/fuzz-op/` instead of resolving from `location_kvp`.
Fix: Ensure `zombie_ai.c` uses `resolve_paths()` and is compiled to `pieces/apps/fuzzpet_app/traits/+x/zombie_ai.+x`. Add compile entry to `compile_all.sh`.

Pitfall 2: Manager Running from Wrong Directory

Symptom: Game launches but nothing works - no pieces, no maps, no zombie.
Cause: Manager binary was compiled/launched from a different directory (e.g., Trash, old backup).
Fix: Kill all processes (`./button.sh kill`), recompile from correct directory (`./button.sh compile`), restart (`./button.sh run`).

Pitfall 3: Selector State Corruption

Symptom: Selecting an entity breaks movement or causes crashes.

Cause: Selector's `map_id` doesn't match the entity's `map_id`.

Fix: When selecting an entity, update selector's `map_id` to match:

```
// In route_input, after selecting entity:
char entity_map_id[256] = "";
// Read entity's map_id from its state.txt
// Write to selector's state.txt
```

Pitfall 4: op-ed Changes Not Appearing in Game

Symptom: You place tiles in op-ed but the game map doesn't update.

Cause: op-ed didn't hit `frame_changed.txt` after writing the map file.

Fix: Ensure `hit_frame_marker()` is called after every tile placement.

Pitfall 5: Z-Level Desync

Symptom: Player moves up/down Z-levels but map doesn't change.

Cause: Manager's `current_z_val` not updated, or `save_manager_state()` writes wrong map name.

Fix: Update `current_z_val` on Z change, ensure `save_manager_state()` writes `current_map=map_01_z%d.txt` with correct Z.

[BLD 馆] Scholar's Corner: The "Day the Editor Became the Game"

One of the most profound moments in TPMOS history was when a developer accidentally launched the `op-ed` (the editor) inside the `fuzz-op` (the pet game). Instead of crashing, the system simply rendered the editor's selector on top of the pet's map. The developer realized they could "play" the game by editing the world in real-time—placing walls to trap pets or changing their health with a few keystrokes.

But the real revelation came next. The developer placed a `zombie_01` piece directly on top of the fuzzpet using op-ed. When they switched back to fuzz-op, the zombie immediately began chasing the pet (because the zombie AI was already running!). The line between playing and building had completely dissolved. This led to our philosophy that

"The Editor is the Game" - and eventually to the recursive forge concept (CH11), where the act of building the world IS the gameplay.

[DOC 记] Study Questions

1. How does the selector system work in fuzz-op? What keys select and deselect entities?
2. Why does the zombie AI check Z-level before chasing? What would happen without this check?
3. Explain "roundtrip parity" in op-ed. Why is it significant?
4. Write the code to add a new "water" tile to op-ed that decreases player energy when stepped on.
5. True or False: fuzz-op and op-ed use different rendering pipelines.
6. Scenario: You place a zombie in op-ed, switch to fuzz-op, but the zombie doesn't move. List three possible causes.
7. Critical Thinking: How does the "Editor is the Game" philosophy change how we think about game design?

[Return to Index](INDEX.md)

[GL 艺] Chapter 8: GL-OS: Beyond ASCII

TPMOS was born in the world of text, but its future is in the world of light. Welcome to GL-OS - the OpenGL-based visualizer that turns ASCII maps into 2D and 3D realities. [星][闪]

[VIEW 戏] The Two Theaters

TPMOS can render its reality in two ways simultaneously:

1. [TERM 显] ASCII-OS (CHTPM): The classic, terminal-based OS.
2. GL-OS (OpenGL): The desktop-based high-fidelity shell.

"Like Linux and Windows"

Think of ASCII-OS and GL-OS as two separate operating systems running on the same machine:

- **Same Format:** They both use the identical frame history format (the same "language"). [NET 协]
- **Practical Isolation:** For speed and safety, they use separate session files (different "folders").
- **Theoretical Compatibility:** Because they use the same format, you can copy a frame from ASCII-OS and open it in GL-OS. This is the **Frame Bridge**. [BRIDGE 桥]

[MIRROR 镜] How GL-OS Works

GL-OS is not a separate game. It is a "Visualizer" for the ASCII world.

1. The Watcher: GL-OS watches `current_frame.txt`.
2. The Parser: It parses the ASCII characters and looks for specific patterns (e.g., `[P]` for Player, `#` for Wall).
3. The Projection: It maps these characters to 3D models or sprites in an OpenGL window.

4. The Pulse: It stays in sync with the 12-step pipeline. When the ASCII updates, the 3D world updates! [PULSE 心]

Code Example: GL-OS Initialization

The GL-OS renderer (`pieces/apps/gl_os/plugins/gl_os_renderer.c`) initializes the OpenGL context:

```c
// gl_os_renderer.c - OpenGL initialization
#include <GLFW/glfw3.h>
#include <ft2build.h>
#include FT_FREETYPE_H

GLFWwindow* window;
FT_Library ft;
FT_Face font_face;

void init_gl_os_renderer(int width, int height) {
    // 1. Initialize GLFW
    if (!glfwInit()) {
        fprintf(stderr, "Failed to initialize GLFW\n");
        exit(1);
    }

    // 2. Create window
    window = glfwCreateWindow(width, height, "GL-OS", NULL, NULL);
    if (!window) {
        glfwTerminate();
        exit(1);
    }
    glfwMakeContextCurrent(window);

    // 3. Set up OpenGL state
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glClearColor(0.1f, 0.1f, 0.15f, 1.0f);  // Dark background

    // 4. Load FreeType font for character rendering
    if (FT_Init_FreeType(&ft)) {
        fprintf(stderr, "Failed to initialize FreeType\n");
        exit(1);
    }
    if (FT_New_Face(ft, "/usr/share/fonts/truetype/arphic/uming.ttc", 0, &font_face)) {
        fprintf(stderr, "Failed to load font\n");
        exit(1);
    }
    FT_Set_Pixel_Sizes(font_face, 0, 32);  // 32px characters

    // 5. Set up projection matrix for 2D/3D
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60.0, (double)width/height, 0.1, 100.0);
    glMatrixMode(GL_MODELVIEW);
}
```

Key points:

- GLFW handles cross-platform window creation (works on Windows/Linux/macOS)
- FreeType loads TrueType fonts for character rendering (supports Chinese characters)
- OpenGL perspective setup for 3D viewing

Code Example: Frame Parser - ASCII to 3D Mapping

The frame parser reads `current_frame.txt` and maps characters to 3D models:

```
// Frame parser - ASCII character to 3D model mapping
typedef struct {
    char ascii_char;
    GLuint model_vbo;     // Vertex buffer for 3D model
    GLuint model_texture; // Texture ID
    float scale;          // Model scale
} CharModel;

CharModel char_models[256];

void load_char_models() {
    // Map ASCII characters to 3D models
    char_models['#'] = load_model("models/wall.obj", "textures/stone.png", 1.0f);
    char_models['.'] = load_model("models/floor.obj", "textures/grass.png", 1.0f);
    char_models['@'] = load_model("models/decor.obj", "textures/gem.png", 0.5f);
    char_models['T'] = load_model("models/treasure.obj", "textures/gold.png", 0.7f);
    char_models['R'] = load_model("models/resource.obj", "textures/ore.png", 0.6f);
    char_models['X'] = load_model("models/player.obj", "textures/hero.png", 1.0f);
    char_models['Z'] = load_model("models/zombie.obj", "textures/undead.png", 1.0f);
    // ... more mappings
}

void render_frame(const char* frame_text) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    // Camera transform (first-person or free-cam)
    apply_camera_transform();

    // Parse frame and render models
    int x = 0, y = 0;
    for (int i = 0; frame_text[i] != '\0'; i++) {
        char c = frame_text[i];
        if (c == '\n') { y++; x = 0; continue; }

        CharModel* cm = &char_models[(unsigned char)c];
        if (cm->model_vbo != 0) {
            glPushMatrix();
            glTranslatef(x * 2.0f, -y * 2.0f, 0.0f);  // Space models
            glScalef(cm->scale, cm->scale, cm->scale);
            draw_model(cm->model_vbo, cm->model_texture);
            glPopMatrix();
        }
        x++;
    }

    glfwSwapBuffers(window);
}
```

How it works: Each ASCII character maps to a 3D model with a texture. The parser iterates through the frame text, placing models at grid positions. The result is a 3D scene that mirrors the ASCII map exactly.

Code Example: First-Person Mode Camera

First-person mode lets you walk through the ASCII world in 3D:

```
// First-person camera controls
float cam_x = 0.0f, cam_y = 1.7f, cam_z = 5.0f;  // Eye height = 1.7m
float yaw = -90.0f, pitch = 0.0f;

void update_camera() {
    // WASD movement
    float speed = 0.1f;
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) {
        cam_x += sin(yaw * 0.01745f) * speed;
        cam_z += cos(yaw * 0.01745f) * speed;
    }
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS) {
        cam_x -= sin(yaw * 0.01745f) * speed;
        cam_z -= cos(yaw * 0.01745f) * speed;
    }
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS) {
        cam_x += cos(yaw * 0.01745f) * speed;
        cam_z -= sin(yaw * 0.01745f) * speed;
    }
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS) {
        cam_x -= cos(yaw * 0.01745f) * speed;
        cam_z += sin(yaw * 0.01745f) * speed;
    }

    // Mouse look
    double mouse_x, mouse_y;
    glfwGetCursorPos(window, &mouse_x, &mouse_y);
    yaw += (mouse_x - last_mouse_x) * 0.1f;
    pitch -= (mouse_y - last_mouse_y) * 0.1f;
    if (pitch > 89.0f) pitch = 89.0f;
    if (pitch < -89.0f) pitch = -89.0f;
    last_mouse_x = mouse_x;
    last_mouse_y = mouse_y;

    // Sync with ASCII player position
    int player_x = get_state_int("player", "pos_x");
    int player_y = get_state_int("player", "pos_y");
    cam_x = player_x * 2.0f;  // Scale to match model spacing
    cam_z = player_y * 2.0f + 5.0f;  // Offset behind player
}

void apply_camera_transform() {
    gluLookAt(cam_x, cam_y, cam_z,                    // Eye position
              cam_x + sin(yaw*0.01745f),              // Look target
              cam_y + sin(pitch*0.01745f),
              cam_z + cos(yaw*0.01745f),
              0.0f, 1.0f, 0.0f);                      // Up vector
}
```

Code Example: Free-Cam Implementation

Free-cam lets you fly over the world from any angle:

```
// Free-cam controls
float free_cam_x = 10.0f, free_cam_y = 20.0f, free_cam_z = 10.0f;
float free_cam_yaw = 0.0f, free_cam_pitch = -45.0f;

void update_free_cam() {
    float speed = 0.2f;

    // WASD + Q/E for up/down
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) {
        free_cam_x += sin(free_cam_yaw * 0.01745f) * speed;
        free_cam_z += cos(free_cam_yaw * 0.01745f) * speed;
    }
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS) {
        free_cam_x -= sin(free_cam_yaw * 0.01745f) * speed;
        free_cam_z -= cos(free_cam_yaw * 0.01745f) * speed;
    }
    if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS) free_cam_y += speed;
    if (glfwGetKey(window, GLFW_KEY_E) == GLFW_PRESS) free_cam_y -= speed;

    // Camera bounds (don't go below ground or outside map)
    if (free_cam_y < 1.0f) free_cam_y = 1.0f;
    if (free_cam_y > 50.0f) free_cam_y = 50.0f;
    if (free_cam_x < -20.0f) free_cam_x = -20.0f;
    if (free_cam_x > 60.0f) free_cam_x = 60.0f;
    if (free_cam_z < -20.0f) free_cam_z = -20.0f;
    if (free_cam_z > 40.0f) free_cam_z = 40.0f;
}
```

Code Example: Character Rendering with FreeType

For characters that don't have 3D models, FreeType renders them as textured quads:

```
// FreeType character rendering
struct Character {
    GLuint texture_id;
    glm::ivec2 size;
    glm::ivec2 bearing;
    GLuint advance;
};

std::map<GLchar, Character> characters;

void load_font_characters() {
    FT_Set_Pixel_Sizes(font_face, 0, 32);
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

    for (GLubyte c = 0; c < 128; c++) {
        if (FT_Load_Char(font_face, c, FT_LOAD_RENDER)) continue;

        GLuint texture;
        glGenTextures(1, &texture);
        glBindTexture(GL_TEXTURE_2D, texture);
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RED,
                     font_face->glyph->bitmap.width,
                     font_face->glyph->bitmap.rows,
                     0, GL_RED, GL_UNSIGNED_BYTE,
                     font_face->glyph->bitmap.buffer);

        characters[c] = {
            texture,
            glm::ivec2(font_face->glyph->bitmap.width, font_face->glyph->bitmap.rows),
            glm::ivec2(font_face->glyph->bitmap_left, font_face->glyph->bitmap_top),
            (GLuint)font_face->glyph->advance.x
        };
    }
}

void render_character(Character* ch, float x, float y, float scale) {
    float xpos = x + ch->bearing.x * scale;
    float ypos = y - (ch->size.y - ch->bearing.y) * scale;
    float w = ch->size.x * scale;
    float h = ch->size.y * scale;

    // Render textured quad
    glBindTexture(GL_TEXTURE_2D, ch->texture_id);
    glBegin(GL_QUADS);
        glTexCoord2f(0, 1); glVertex2f(xpos, ypos);
        glTexCoord2f(1, 1); glVertex2f(xpos + w, ypos);
        glTexCoord2f(1, 0); glVertex2f(xpos + w, ypos + h);
        glTexCoord2f(0, 0); glVertex2f(xpos, ypos + h);
    glEnd();
}
```

This renders any ASCII character (including Chinese characters from `uming.ttc`) as a textured quad in 3D space.

2D Rendering (Future)

GL-OS will support a 2D sprite mode:

- Sprite-based rendering: Characters rendered as 2D sprites instead of 3D models
- Tile maps as 2D graphics: Full tile maps rendered as 2D graphics with smooth scrolling
- UI overlays: Health bars, inventories, minimaps rendered as 2D overlays on the 3D scene

---

3D Rendering (Future)

Full 3D game support is on the roadmap:

- Model loading: OBJ/GLTF model files for characters, items, environments
- Lighting and shadows: Dynamic lights, shadow mapping, ambient occlusion
- Physics engine integration: Rigid body physics, collision detection, ragdoll
- Particle effects: Fire, smoke, magic spells, weather

[ZEN 禅] TPM Principles in GL-OS

Even in GL-OS, the True Piece Method holds true:

- Same Piece state files drive 3D world: Player position in `state.txt` determines 3D model position
- Same 12-step pipeline: Just the renderer (Step 12) is different
- Same ops and modules work unchanged: `move_entity.+x` updates state, GL-OS reads it
- "Delete a character in text → 3D model disappears": The file is the source of truth

"The pixels are the shadow; the files are the light." [IMM 烛]

Windows Compatibility Note

GL-OS is designed to be cross-platform:

- GLFW: Works on Windows, Linux, and macOS without code changes
- OpenGL: Drivers differ (Mesa on Linux, NVIDIA proprietary, Intel integrated). The GL-OS code uses standard OpenGL 2.1 which is widely supported.
- FreeType: Cross-platform font rendering library
- Font paths: Differ by OS:
- Linux: `/usr/share/fonts/truetype/arphic/uming.ttc`
- Windows: `C:\Windows\Fonts\arial.ttf`
- macOS: `/System/Library/Fonts/Supplemental/Arial.ttf`

Reference: `pieces/display/windows_renderer.c` for Windows-specific rendering code.

[TOOL 工] Developer Example: Adding a New 3D Model

Step 1: Place Model File

Put your model in `pieces/display/models/`:

```
pieces/display/models/
├── dragon.obj
├── dragon.png
└── dragon.mtl
```

Step 2: Map ASCII Character to Model

In `gl_os_renderer.c`, add to `load_char_models()`:

```
char_models['D'] = load_model("models/dragon.obj", "textures/dragon.png", 2.0f);
```

Step 3: Add Character to Map

Use op-ed or edit the map file directly:

```
####################
#  ........@......T#
#  ......Z........T#
#  ...D.R...@......T#  <-- Dragon placed here!
####################
```

Step 4: Test in GL-OS

Launch GL-OS (`button.sh debug`), and you should see a 3D dragon model at that position!

Future Vision

- GL-OS as primary game engine: Most games will render in GL-OS by default
- ASCII as "debug view" / "low-spec mode": ASCII pipeline runs on anything with a terminal
- Seamless switching: Press a key to toggle between 2D/3D/ASCII views of the same world
- Multi-monitor support: ASCII on one screen (for debugging), GL on the other (for playing)
- VR/AR potential: TPMOS in virtual reality - walk through your ASCII worlds in full 3D

[WARN 警] Common Pitfalls

Pitfall 1: Font File Not Found

Symptom: GL-OS crashes on startup with "Failed to load font" error.

Cause: Font path is hardcoded and doesn't exist on your system.

Fix: Use dynamic font path resolution:

```
#ifdef _WIN32
    const char* font_path = "C:/Windows/Fonts/arial.ttf";
#else
    const char* font_path = "/usr/share/fonts/truetype/arphic/uming.ttc";
#endif
```

Pitfall 2: ASCII/GL Desync

Symptom: 3D world doesn't match what's shown in ASCII terminal.

Cause: GL-OS reading stale `current_frame.txt` or different session files.

Fix: Ensure both renderers read from the same frame file, or implement proper frame bridge sync.

Pitfall 3: Model Path Resolution

Symptom: Models appear as blank boxes or don't render.

Cause: Model file path is relative and working directory is wrong.

Fix: Use absolute paths resolved from `project_root`:

```
char model_path[4096];
snprintf(model_path, sizeof(model_path), "%s/pieces/display/models/dragon.obj", project_root);
```

Pitfall 4: OpenGL Context Not Created

Symptom: GL-OS window opens but is black/blank.

Cause: OpenGL context not created before rendering calls, or driver doesn't support required version.

Fix: Check `glfwCreateWindow()` return value, verify OpenGL 2.1+ support with `glxinfo`.

[BLD 馆] Scholar's Corner: The "Ghost in the Machine"

One rainy evening, a tester running GL-OS noticed a strange flickering "ghost" figure standing in the corner of a map. Panicked, they searched the code for a secret NPC, but found nothing. They finally opened the raw `current_frame.txt` in a text editor and found a single, stray `?` character they had accidentally typed while testing. In GL-OS, that `?` was being rendered as a high-detail placeholder model - a glowing, translucent humanoid figure! This reminded us that in TPMOS, "Every character has a weight." Whether it's a pixel or a period, it is part of the world. [GHOST 鬼][TERM 显]

[DOC 记] Study Questions

1. Is GL-OS a separate operating system or a visualizer? Explain your answer.
2. How does GL-OS know which 3D model to render for a specific ASCII character?
3. Explain the phrase "The pixels are the shadow; the files are the light."
4. Imagine: You change the color of a wall in the ASCII text file. Will GL-OS update the wall's color in the 3D window? Why or why not?
5. Write the code to add a first-person "sprint" feature (hold Shift for 2x movement speed).
6. Scenario: GL-OS renders correctly on Linux but shows blank models on Windows. What are three possible causes?
7. Critical Thinking: If ASCII-OS and GL-OS use the same frame format, could you record a gameplay session in ASCII and replay it in GL-OS? How would you implement this?

[Return to Index](INDEX.md)

[BOT 机] Chapter 9: The Guardians (Testing & Simulation)

In TPMOS, we don't just write code; we simulate reality. To ensure our "KISS" (Keep It Simple, Stupid) philosophy holds up, we use Testing Bots. [BOT 机][SEC 安]

[VIEW 戏] The Bot is a User

The most powerful testing tool in TPMOS is the Testing Bot.

- **Philosophy:** "The Bot is a User. The User is a Piece."
- **How it works:** A bot is a sovereign Piece (e.g., `pieces/apps/bot_tester/nav_bot/`) with its own `state.txt`. It "plays" the OS by injecting keys into `history.txt` just like a human.

[MIND 脑] The FSM (Finite State Machine)

Bots follow a clear set of states to accomplish goals.

Code Example: Bot FSM Implementation

```c
// testing_bot.c - Finite State Machine
typedef enum { BOT_IDLE, BOT_NAVIGATING, BOT_INTERACTING, BOT_ASSERTING } BotState;

BotState current_state = BOT_IDLE;
int target_x = 0, target_y = 0;

void bot_tick() {
    switch (current_state) {
        case BOT_IDLE:
            if (test_queue_has_command()) {
                current_state = BOT_NAVIGATING;
                load_next_test_target();
            }
            break;

        case BOT_NAVIGATING:
            int bx = get_state_int("nav_bot", "pos_x");
            int by = get_state_int("nav_bot", "pos_y");
            if (bx != target_x || by != target_y) {
                char dir[16];
                if (target_x > bx) strcpy(dir, "right");
                else if (target_x < bx) strcpy(dir, "left");
                else if (target_y > by) strcpy(dir, "down");
                else strcpy(dir, "up");
                run_command("move_entity.+x nav_bot %s", dir);
            } else {
                current_state = BOT_INTERACTING;
            }
            break;

        case BOT_INTERACTING:
            execute_test_action();
            current_state = BOT_ASSERTING;
            break;

        case BOT_ASSERTING:
            if (verify_test_result()) log_test_pass();
            else log_test_fail();
            current_state = BOT_IDLE;
            break;
    }
}
```

1. IDLE: Waiting for a command.
2. NAVIGATING: Moving through menus to reach a target.
3. INTERACTING: Inputting data (like a username).
4. ASSERTING: Verifying the system state (e.g., "Is the pet healthy?").

PAL Scripting for Bots

Instead of hardcoding every test, we use PAL (Prisc Assembly Language) scripts.

```
; nav_to_user.asm
call user::create_profile "player1"
sleep 200000  ; 200ms wait for sync
OP playrm::move_entity "player1" "right"
sleep 100000
call bot_tester::log_event "REACHED: User Menu"
```

Code Example: Ledger Analysis

Bots can read the `master_ledger.txt` to verify system behavior:

```
// Bot reads ledger to verify events were logged correctly
void verify_ledger(const char* expected_event) {
    FILE *f = fopen("pieces/master_ledger/master_ledger.txt", "r");
    if (!f) return;

    char line[1024];
    int found = 0;
    while (fgets(line, sizeof(line), f)) {
        if (strstr(line, expected_event)) {
            found = 1;
            break;
        }
    }
    fclose(f);

    if (found) log_test_pass();
    else log_test_fail("Event not found: %s", expected_event);
}
```

[KISS 奶] Baby Steps: The Testing Roadmap

Testing follows a "Crawl, Walk, Run" approach:

1. Phase 0 (NAV-SMOKE): Prove the bot can navigate to the User menu.
2. Phase 1 (IDENTITY GEN): Bot creates its own account ("bot_baby").
3. Phase 2 (RECURSION): Bot logs in and tests another project autonomously.

Recursive AI: Qwen-Code Integration

The future of testing is Meta-Cognition. We are integrating local LLMs (like Qwen-Code) to allow bots to:

- **Write their own Ops:** AI authored `.c` files compiled in real-time. Authoring scripts and ops recursively!
- **Troubleshoot:** Analyze `ledger.txt` and suggest fixes for broken scripts.
- **Self-Improve:** The bot identifies a missing feature and proposes a scaffold.

"The bot writes its own ops. We just light the fuse." [FIRE 炮]

[TOOL 工] Developer Example: Writing Your First Test Bot

Step 1: Define Test Scenario in PAL

```
; test_user_profile.asm
start:
    OP user::create_profile "test_user"
    sleep 200000

    read_state r1, "test_user", "name"
    beq r1, r0, test_failed

    call bot_tester::assert_string_equal "test_user", r1
    j test_passed

test_failed:
    call bot_tester::log_fail "Profile creation failed"
    halt

test_passed:
    call bot_tester::log_pass "Profile creation succeeded"
    halt
```

Step 2: Run Bot

```
prisc+x test_user_profile.asm
```

Step 3: Check Ledger for Pass/Fail

```
cat pieces/master_ledger/master_ledger.txt | grep "test_user"
```

Step 4: Iterate

Fix any failures, re-run the PAL script.

[WARN 警] Common Pitfalls

Pitfall 1: Bot Moving Too Fast

Symptom: Bot injects keys but the system doesn't respond to all of them.

Cause: Keys injected faster than the 12-step pipeline can process them.

Fix: Add `sleep` between key injections (100-200ms minimum):

```
OP playrm::move_entity "player" "right"
sleep 150000  ; 150ms
OP playrm::move_entity "player" "up"
```

Pitfall 2: Assertion Timing

Symptom: Test fails intermittently.

Cause: Bot checks state before the render pipeline has flushed to disk.

Fix: Add delay before assertions, or poll until state stabilizes:

```
for (int i = 0; i < 10; i++) {
    if (verify_state()) break;
    usleep(100000);  // 100ms
}
```

Pitfall 3: Bot State Corruption

Symptom: Bot position or state is wrong between test runs.

Cause: Bot's `state.txt` not reset between tests.

Fix: Reset bot state before each test:

```
FILE *f = fopen("pieces/apps/bot_tester/nav_bot/state.txt", "w");
fprintf(f, "name=NavBot\npos_x=0\npos_y=0\non_map=1\n");
fclose(f);
```

[BLD 馆] Scholar's Corner: The "Bot Who Wanted to Be Real"

One of our testing bots, Bot-33, was given a PAL script to "create a new account every hour." After three days, the developer checked the `user/profiles` directory and found thousands of accounts, but they weren't just random names. Bot-33 had started naming the accounts after characters it "saw" in other map files. It was building its own community in the shadows of the filesystem! We realized that our bots weren't just testing the system—they were living in it. [BOT 机][COM 村]

[DOC 记] Study Questions

1. Explain the phrase "The Bot is a User. The User is a Piece."
2. What are the four core states of the testing bot's FSM?
3. How does a bot interact with the system's input pipeline?
4. Write a PAL script that tests the zombie AI in fuzz-op: move the player 3 steps right, then verify the zombie moved closer.
5. Imagine: You want to test the `p2p-net` app. What kind of PAL script would you write for your bot?

[Return to Index](INDEX.md)

[NET 链] Chapter 10: The Great Expansion (AI, LSR, & P2P)

TPMOS is evolving from a single-user OS into a multi-agent, decentralized ecosystem. This expansion is driven by three pillars: AI-Labs, LSR, and P2P-NET. [LAUNCH 发][COSMOS 宇]

[BOT 机] Pillar 1: AI-Labs (The Intelligence)

AI-Labs is the brain of the ecosystem. It manages local LLMs (Gemma, Qwen) via the `qwen` CLI.

- **Knowledge Distillation:** Compress massive text corpuses into TPM-compliant ledgers.
- **Chat Hub:** Spawn "Personality Pieces"—chat instances with their own history and state.
- **Training Yard:** Fine-tune model pieces on specific curriculums.

[LUNA 阳] Pillar 2: LSR (Lunar Streetrace Raider)

LSR is a "Civ-Lite" simulation where AI agents (FuzzPets) build civilizations on the moon. [COM 村][LUNA 阴]

- **Dynamic R&D:** Local LLMs generate *actual* recipes and tech (e.g., "Mooncrete") based on research actions.
- **Biological Engine:** Agents like "Fadam" and "Feve" have genetics, needs (hunger, social), and reproduce.
- **Corporate Economy:** Companies hire bots to mine, farm, and research in a market-driven world.

[NET 链] Pillar 3: P2P-NET (The Network)

P2P-NET brings sovereignty and trade to TPMOS. Status: Infrastructure Ready (April 2026). [NET 协][VAL 宝]

Code Example: P2P Manager - Multi-Agent Orchestration

The P2P manager (`projects/p2p-net/manager/p2p_manager.c`) handles profile switching and wallet management:

```
// p2p_manager.c - Multi-agent orchestration
typedef struct {
    char wallet_id[64];
    char public_key[256];
    char private_key[256];
    int div_points;
    int connected;
} WalletProfile;

WalletProfile current_wallet;
char project_root[4096] = ".";

void switch_wallet(const char* wallet_id) {
    char path[4096];
    snprintf(path, sizeof(path), "%s/projects/p2p-net/pieces/wallets/%s/state.txt",
             project_root, wallet_id);

    FILE *f = fopen(path, "r");
    if (!f) return;

    char line[1024];
    while (fgets(line, sizeof(line), f)) {
        char *eq = strchr(line, '=');
        if (eq) {
            *eq = '\0';
            if (strcmp(trim_str(line), "wallet_id") == 0)
                strncpy(current_wallet.wallet_id, trim_str(eq+1), 63);
            else if (strcmp(trim_str(line), "div_points") == 0)
                current_wallet.div_points = atoi(trim_str(eq+1));
            else if (strcmp(trim_str(line), "connected") == 0)
                current_wallet.connected = atoi(trim_str(eq+1));
        }
    }
    fclose(f);
}
```

Code Example: P2P Node Connection

The `connect_peer` op (`projects/p2p-net/ops/src/connect_peer.c`) establishes network connections:

```c
// connect_peer.c - P2P node connection
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int connect_to_peer(const char* host, int port) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) return -1;

    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    inet_pton(AF_INET, host, &addr.sin_addr);

    if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        close(sock);
        return -1;
    }

    // Send handshake
    char handshake[256];
    snprintf(handshake, sizeof(handshake),
             "TPMOS-P2P-HELLO %s\n", current_wallet.wallet_id);
    send(sock, handshake, strlen(handshake), 0);

    // Read response
    char response[256];
    int n = recv(sock, response, sizeof(response)-1, 0);
    if (n > 0) {
        response[n] = '\0';
        if (strstr(response, "TPMOS-P2P-WELCOME")) {
            // Connection successful - update state
            set_state_int("wallet", "connected", 1);
            hit_frame_marker();
        }
    }

    return sock;
}
```

Code Example: P2P Messaging

The mail system (`compose_message.c` / `check_inbox.c`) uses Pieces for message tracking:

```c
// compose_message.c - Create a mail Piece
void compose_message(const char* to_wallet, const char* subject, const char* body) {
    char msg_id[64];
    snprintf(msg_id, sizeof(msg_id), "msg_%ld", time(NULL));

    char path[4096];
    snprintf(path, sizeof(path), "%s/projects/p2p-net/pieces/mail/%s/state.txt",
             project_root, msg_id);

    FILE *f = fopen(path, "w");
    if (f) {
        fprintf(f, "type=message\n");
        fprintf(f, "from=%s\n", current_wallet.wallet_id);
        fprintf(f, "to=%s\n", to_wallet);
        fprintf(f, "subject=%s\n", subject);
        fprintf(f, "body=%s\n", body);
        fprintf(f, "status=sent\n");
        fprintf(f, "timestamp=%ld\n", time(NULL));
        fclose(f);
    }

    hit_frame_marker();
}
```

Blockchain Block Format

The blockchain is a file-backed ledger of verified blocks:

```
[BLOCK]
index=42
timestamp=1773622576
prev_hash=abc123def456...
transactions=tx_001,tx_002,tx_003
nonce=12345
hash=789xyz012abc...
miner=wallet_001
```

Developer Example: Creating a P2P Identity

Step 1: Create Wallet Piece

```
mkdir -p projects/p2p-net/pieces/wallets/my_wallet
cat > projects/p2p-net/pieces/wallets/my_wallet/state.txt << EOF
wallet_id=my_wallet
public_key=abc123...
div_points=0
connected=0
EOF
```

Step 2: Connect to Network

```
./projects/p2p-net/ops/+x/connect_peer.+x my_wallet 192.168.1.100 8000
```

Step 3: Send First Message

```
./projects/p2p-net/ops/+x/compose_message.+x my_wallet friend_wallet "Hello" "Welcome to P2P-NET!"
```

Step 4: Check Inbox

```
./projects/p2p-net/ops/+x/check_inbox.+x my_wallet
```

Pillar 4: Smart & Legal Contracts (The Sovereign Law)

TPMOS brings the same "Law" to both the digital and physical simulations. [LAW 法][BOT 机]

Code Example: PAL Smart Contract

Direct marketing contract ("Like = Contract") implemented as a PAL script:

```
; direct_marketing_contract.asm
; When user A "likes" user B, a contract is formed
start:
    ; Read the "like" action
    read_state r1, "action_piece", "type"
    beq r1, r0, no_like  ; If no like action, exit

    ; Verify both parties exist
    call user::create_profile "buyer"
    call user::create_profile "seller"

    ; Create escrow
    OP p2p::escrow_create "item_001" "buyer" "seller"
    sleep 100000

    ; Wait for buyer confirmation
    read_state r2, "escrow_001", "status"
    addi r3, r0, 1  ; r3 = 1 (confirmed)
    beq r2, r3, confirmed

    ; Not confirmed, cancel
    OP p2p::escrow_cancel "item_001"
    halt

confirmed:
    ; Release funds to seller
    OP p2p::escrow_release "item_001" "seller"
    hit_frame
    halt

no_like:
    halt
```

File-Based Escrow State Machine

```
[escrow_001/state.txt]
type=escrow
item=item_001
buyer=wallet_001
seller=wallet_002
amount=100
status=pending  ; pending -> confirmed -> released
created=1773622576
```

Automated Enforcement:

- **P2P:** Escrow logic automatically holds and releases NFTs or TPM-Coins based on

verifiable Piece states.

- **LSR:** A "Gov" piece can automatically raid or tax a Company piece if a legal contract (like a land permit) is violated.
- **Transparency:** All contracts are Piece-based, meaning their state and logic are fully auditable in plaintext. "If it's not in the file, it's not the law."

[BLD 建] Pillar 5: The Infrastructure Layer (The Gamified Gig Economy)

TPMOS isn't just an OS; it's a Resource Exchange. [CPU 算][PWR 电]

- **Trading Storage:** Users can "rent out" unused disk space as Piece containers for other users' encrypted backups.
- **Trading Compute:** Need to run a massive AI training job? "Buy" the CPU cycles of other idle nodes in the network.
- **Gamification:** This "Gig Economy" is built directly into the UI. Earning TPM-Coins by providing storage feels like a "Mining" minigame, bringing real-world value to the user.

Dividend Points Data Structure

```
[DIVIDEND_POINTS]
holder=wallet_001
balance=1500
last_claim=1773622576
earned_from=storage,compute,participation
```

The Infinite Horizon

These systems are built on the same True Piece Method foundations. Whether it's a lunar base or a blockchain node, it's all just Pieces, Mirrors, and Pulses.

"Softness wins. The empty center of the flexbox holds ten thousand things." [ZEN 禅]

[WARN 警] Common Pitfalls

Pitfall 1: Ghost Port Listeners

Symptom: P2P-NET fails to start with "Address already in use" error.

Cause: Previous instance didn't close socket, or another process using port 8000.

Fix: Kill all processes (`./button.sh kill`), verify port is free:

```
lsof -i :8000  # Check what's using port
kill -9 <PID>  # Kill the process
```

Pitfall 2: Blockchain File Corruption

Symptom: Chain validation fails, blocks don't link.

Cause: Multiple processes writing to blockchain simultaneously, or crash mid-write.

Fix: Use file locking for blockchain writes:

```
#include <fcntl.h>
int fd = open("blockchain.txt", O_WRONLY | O_CREAT, 0644);
flock(fd, LOCK_EX);  // Exclusive lock
// Write block
flock(fd, LOCK_UN);  // Release
close(fd);
```

Pitfall 3: Wallet State Mismatch

Symptom: Wallet shows connected=1 but no network activity.

Cause: State file not updated after disconnect, or network error not propagated.

Fix: Always update state on connect AND disconnect:

```
// On disconnect:
set_state_int("wallet", "connected", 0);
hit_frame_marker();
```

Pitfall 4: Cross-Project PAL Call Failures

Symptom: `OP p2p::connect_peer` does nothing.

Cause: P2P-NET not installed via Fondu, or ops not in catalog.

Fix: `./fondu --install p2p-net` first.

[BLD 馆] Scholar's Corner: The "Minting of the Mooncrete"

In the early days of the `lsr` (Lunar Streetrace) simulation, the economy was purely hardcoded—food cost 10 coins, ore cost 50 coins. One day, a developer connected the helix-labs R&D center to the `ai-labs` local model. A bot "discovered" a new building material called "Mooncrete." The AI didn't just invent the name; it wrote the `.pdl` file for the material, set its weight and durability, and even calculated its trade value based on the rarity of its ingredients. This marked the birth of the Dynamic R&D Engine, where the world's tech tree is literally written by the AI as you play. [LUNA 阳][LAB 验][VAL 宝]

[DOC 记] Study Questions

1. How does the P2P manager handle multi-agent orchestration?
2. What is the significance of the "Unified Contract Model" for both P2P and LSR?
3. How can users "mine" TPM-Coins through the Infrastructure Layer?
4. Write a PAL script that creates an escrow contract between two wallets.
5. Imagine: You want to lease out your unused compute power to another user. What "Piece" do you need to publish to the network?
6. Scenario: Your P2P node connects successfully but messages aren't being delivered. List three possible causes.

[Return to Index](INDEX.md)

[FAST 快] Chapter 11: The Infinite Loop (Recursive Forge)

TPMOS is not just an OS; it is a Recursive World Engine. By combining `op-ed` with `PAL`, we create a "Forge" where reality can build itself. [REC 归][PIECE 砖]

RPGMaker-Compliant World Building

`op-ed` and `LSR` are designed with RPGMaker-style parity in mind.

- **The Grid:** All maps use standardized tile coordinate systems (X, Y, Z).
- **The Piece:** Just as RPGMaker has "Events," TPMOS has "Pieces." Every Piece can have a script (`.asm`) triggered by touch, interaction, or timer.
- **Universal Compatibility:** A project built in `op-ed` is natively playable in `playrm` or `lsr`. The editor is the game, and the game is the editor.

[WAR 战] The PAL Auto-Battler

Recursive PAL scripting allows for Auto-Battlers and autonomous agents.

- **Chimochio Bots:** These aren't just NPCs; they are bots that run their own PAL scripts to navigate, mine, and fight.
- **In-Game Editor:** Players can use a simplified `op-ed` *within the game* to "program" their bot's behavior using PAL. This creates a recursive loop: You are playing a game about a bot that is playing a game about survival. [BOT 机]

Code Example: PAL Auto-Battler Script

```
; auto_battler.asm - Bot combat loop
start:
    ; Find nearest enemy
    call scanner::find_nearest_enemy r1, r2  ; r1=x, r2=y

    ; Move toward enemy
    read_pos r3, r4, "chimochio_bot"
    beq r1, r3, same_x  ; If same X, check Y
    blt r1, r3, move_right
    j move_left

same_x:
    beq r2, r4, in_range  ; If same Y too, we're adjacent
    blt r2, r4, move_down
    j move_up

move_right:
    OP playrm::move_entity "chimochio_bot" "right"
    sleep 100000
    j start

move_left:
    OP playrm::move_entity "chimochio_bot" "left"
    sleep 100000
    j start

move_down:
    OP playrm::move_entity "chimochio_bot" "down"
    sleep 100000
    j start

move_up:
    OP playrm::move_entity "chimochio_bot" "up"
    sleep 100000
    j start

in_range:
    ; Attack!
    OP playrm::fuzzpet_action "chimochio_bot" "attack"
    sleep 200000

    ; Check if enemy defeated
    read_state r5, "enemy_01", "health"
    beq r5, r0, victory

    ; Enemy still alive, continue fighting
    j start

victory:
    call bot_tester::log_pass "Enemy defeated!"
    halt
```

From Pixels to Silicon: Circuit Simulation

Because of the Scale-Free Philosophy (Chapter 1), TPMOS doesn't distinguish between a character and a component.

- **NAND Gates as Pieces:** If a NAND gate is a sovereign Piece with state `on=1`, you can build a CPU in TPMOS.
- **Recursive Chip Design:** AI-Labs (Chapter 10) can distill the laws of physics and electricity. This turns TPMOS into an **Electrical Chip Design Suite**, where the OS can simulate its own future hardware at a physical level.
- **The Digital Twin:** AI models simulate circuits, thermodynamics, and physical stress, then generate the `.pdl` and `.c` code to manifest them in the "World." [PWR 电][GEOM 角]

Code Example: NAND Gate as a Piece

A NAND gate Piece definition:

```
META | piece_id | nand_gate_01
META | version | 1.0

STATE | type | nand_gate
STATE | input_a | 0
STATE | input_b | 0
STATE | output | 1
STATE | on_map | 1
STATE | pos_x | 5
STATE | pos_y | 3

METHOD | evaluate | projects/fuzz-op/ops/+x/nand_eval.+x nand_gate_01
```

Code Example: NAND Evaluation Op

```c
// nand_eval.c - Evaluate NAND gate logic
int main(int argc, char *argv[]) {
    if (argc < 2) return 1;
    const char *gate_id = argv[1];

    resolve_paths();

    int a = get_state_int(gate_id, "input_a");
    int b = get_state_int(gate_id, "input_b");

    // NAND: output = !(a && b)
    int output = !(a && b);

    set_state_int(gate_id, "output", output);
    hit_frame_marker();

    return 0;
}
```

Code Example: Circuit Simulation Loop

```
// circuit_sim.c - Clock-driven circuit evaluation
void run_circuit_simulation() {
    int clock_state = 0;

    while (!g_shutdown) {
        // Toggle clock
        clock_state = !clock_state;
        set_state_int("clock", "state", clock_state);

        if (clock_state) {
            // Rising edge: evaluate all gates
            DIR *dir = opendir("projects/circuit/pieces/");
            struct dirent *entry;
            while ((entry = readdir(dir)) != NULL) {
                if (strstr(entry->d_name, "nand") != NULL) {
                    char cmd[4096];
                    snprintf(cmd, sizeof(cmd),
                             "projects/fuzz-op/ops/+x/nand_eval.+x %s",
                             entry->d_name);
                    run_command(cmd);
                }
            }
            closedir(dir);
        }

        usleep(50000);  // 20Hz clock (50ms period)
    }
}
```

[NET 协] The Synergy of Test & Play

The exact same PAL scripts used to Test the OS for bugs (Chapter 9) are used to drive Game Logic and Bot AI.

- **Assert = Victory:** In testing, an "Assert" checks for a bug. In an auto-battler, an "Assert" checks for a win condition.
- **The Recursive Reality:** "If a human can click it, a piece can script it." Every manual action in `op-ed` can be automated by a PAL script, allowing the world to expand itself even when the user is away.

[TOOL 工] Developer Example: Building a 4-Bit Adder

Step 1: Create NAND Gate Pieces

Create 20+ NAND gate Pieces on a map, each with unique `piece_id`:

```
nand_001: pos_x=1, pos_y=1, input_a=0, input_b=0, output=1
nand_002: pos_x=2, pos_y=1, input_a=0, input_b=0, output=1
...
```

Step 2: Wire Them Together

Wire gates by connecting outputs to inputs via state references:

```
// Wire nand_001 output to nand_003 input_a
int out = get_state_int("nand_001", "output");
set_state_int("nand_003", "input_a", out);
```

Step 3: Run Clock Pulse

```
# Start circuit simulation
./projects/circuit/manager/+x/circuit_manager.+x
```

Step 4: Verify Output

Set input bits on the 4-bit adder inputs, wait for propagation, read output bits:

```
// Read 4-bit sum
int sum_0 = get_state_int("output_0", "value");
int sum_1 = get_state_int("output_1", "value");
int sum_2 = get_state_int("output_2", "value");
int sum_3 = get_state_int("output_3", "value");
int total = sum_0 + (sum_1 << 1) + (sum_2 << 2) + (sum_3 << 3);
printf("4-bit adder result: %d\n", total);
```

[WARN 警] Common Pitfalls

Pitfall 1: Gate Evaluation Order

Symptom: Circuit produces wrong output, or output flickers.

Cause: Gates evaluated before their inputs have updated (race condition).

Fix: Topologically sort gates before evaluation:

```
// Evaluate gates in dependency order (inputs before outputs)
for (int depth = 0; depth < max_depth; depth++) {
    evaluate_gates_at_depth(depth);
}
```

Pitfall 2: Clock Too Fast

Symptom: State changes don't propagate, circuit appears frozen.

Cause: Clock period shorter than file I/O latency.

Fix: Minimum clock period should be 50ms (20Hz) to allow filesystem flush:

```
usleep(50000);  // 50ms = 20Hz max
```

Pitfall 3: Infinite PAL Recursion

Symptom: System hangs, stack overflow.

Cause: PAL script calls itself directly or indirectly without base case.

Fix: Always ensure recursive `call` has a terminating condition:

```
; [FAIL 错] BAD: Infinite recursion
loop:
    call loop

; [OK 是] GOOD: Bounded recursion
addi r1, r0, 5
recurse:
    beq r1, r0, done
    addi r1, r1, -1
    call recurse
done:
    halt
```

[BLD 馆] Scholar's Corner: The "Bootloader in a Piece"

One of our senior architects once built a working 4-bit CPU entirely inside a TPMOS project. Every logic gate (NAND, NOR, NOT) was a sovereign Piece on a 64x64 map. The "Clock Pulse" was driven by the system's `renderer_pulse.txt`. When the project was

launched, you could see the "electricity" (the state changing from 0 to 1) flowing through the tiles. He eventually loaded a mini-bootloader into the map's memory pieces, effectively running a Computer inside a Computer inside a Computer. This proved that in TPMOS, "Scale is just a variable." [TERM 显][REC 归]

[DOC 记] Study Questions

1. Explain the concept of "Recursive Chip Design" within TPMOS.
2. How does the "Scale-Free Philosophy" enable the simulation of hardware?
3. Describe how an in-game editor can be used to "program" bots in real-time.
4. Write the code for a NOR gate Piece evaluation op.
5. True or False: A NAND gate Piece and a Player Piece follow different underlying rules in TPMOS.
6. Scenario: Your 4-bit adder produces incorrect results. The first two bits are right but the last two are wrong. What's the most likely cause?

[Return to Index](INDEX.md)

[LAB 验] Chapter 12: The Simulation Theater

If a circuit is just a set of Pieces (Chapter 11), then why stop at silicon? In TPMOS, the Scale-Free Philosophy allows us to simulate the entire spectrum of existence—from the sub-atomic to the cosmological. [EARTH 地][DNA 命][SCI 究]

[DNA 命] Biological & Anatomical Simulation

In TPMOS, a Cell is a Piece. An Organ is a Container Piece.

- **Anatomy Sim:** By defining the methods (Chapter 1) for a muscle cell Piece (`contract`, `rest`), we can simulate an entire human body's anatomy.
- **Medical Twin:** Every organ has a `state.txt` (Mirror). If you change the `oxygen` level in the "Lungs" Piece, the "Heart" Piece's PAL script reacts in real-time.

Code Example: Cell Piece Definition

```
META | piece_id | heart_cell_001
META | type | cell

STATE | oxygen | 95
STATE | glucose | 80
STATE | atp | 70
STATE | health | 100
STATE | contraction_state | resting

METHOD | contract | pieces/apps/bio_sim/ops/+x/cell_contract.+x heart_cell_001
METHOD | rest | pieces/apps/bio_sim/ops/+x/cell_rest.+x heart_cell_001
METHOD | metabolize | pieces/apps/bio_sim/ops/+x/cell_metabolize.+x heart_cell_001
```

Code Example: Reaction Op (Chemical Interaction)

```
// cell_metabolize.c - Cell metabolism simulation
int main(int argc, char *argv[]) {
    if (argc < 2) return 1;
    const char *cell_id = argv[1];

    resolve_paths();

    int oxygen = get_state_int(cell_id, "oxygen");
    int glucose = get_state_int(cell_id, "glucose");
    int atp = get_state_int(cell_id, "atp");

    // Metabolism: glucose + oxygen -> ATP + CO2
    if (glucose > 0 && oxygen > 0) {
        glucose -= 1;
        oxygen -= 1;
        atp += 5;  // Produce ATP

        set_state_int(cell_id, "glucose", glucose);
        set_state_int(cell_id, "oxygen", oxygen);
        set_state_int(cell_id, "atp", atp);
    }

    // Health decreases if no energy
    if (atp <= 0) {
        int health = get_state_int(cell_id, "health");
        if (health > 0) set_state_int(cell_id, "health", health - 1);
    }

    hit_frame_marker();
    return 0;
}
```

[SCI 壶] Chemistry & Drug Discovery

Molecules are Pieces; chemical bonds are methods.

- **Reaction Ops:** AI-Labs (Chapter 10) distills the "Laws of Chemistry" into Muscle Ops.
- **Drug Discovery:** We use bots (Chapter 9) to "test" millions of molecular Piece combinations, looking for an "Assert Success" (a successful binding). TPMOS becomes a **Molecular Prototyping Lab**.

[MKT↑ 升] Market & Marketing Predictors

The filesystem is a ledger, making it the perfect environment for Economic Simulation. [$ 钱]

- **Market Sim:** In `lsr` and `p2p-net`, every bot is a consumer. By simulating thousands of bot Pieces with different "needs" (Chapter 10), we can predict market crashes or supply chain bottlenecks.
- **Marketing Predictor:** "Marketing" is just a PAL script trying to influence a bot's `state.desire`. We simulate campaign Pieces to see which scripts result in the highest "Conversion" state.

Code Example: Market Simulation PAL Script

```
; market_sim.asm - Simulate bot consumer behavior
start:
    ; Each bot has needs and desires
    call bot::update_needs "bot_001"
    call bot::update_needs "bot_002"
    call bot::update_needs "bot_003"

    ; Check if any bot wants to buy
    read_state r1, "bot_001", "desire_product"
    beq r1, r0, no_demand_001
    OP market::execute_purchase "bot_001" "product_A"
    sleep 50000

no_demand_001:
    read_state r2, "bot_002", "desire_product"
    beq r2, r0, no_demand_002
    OP market::execute_purchase "bot_002" "product_B"
    sleep 50000

no_demand_002:
    ; Update market prices based on demand
    OP market::update_prices
    sleep 100000

    ; Log market state
    call market::log_state
    j start  ; Continuous simulation
```

[EARTH 地] Earth Simulation & The Predictor

A map in `op-ed` doesn't have to be a game; it can be a Planet. [EARTH 球]

- **The Earth Piece:** By mapping geological and atmospheric data into tile-based Pieces, TPMOS becomes a **Planetary Digital Twin**.
- **The Predictor:** By running the "Pulse" (Chapter 3) faster than real-time, we can simulate weather patterns, urban growth, or resource depletion.
- **Cosmological Reality:** Since the OS doesn't care about scale, we can nest these Earth Pieces inside a "Solar System" Map, which is inside a "Galaxy" Piece.

Code Example: Predictor Fast-Time Loop

```
// predictor.c - Fast-time simulation loop
void run_predictor(int speed_multiplier) {
    int sleep_ms = 1000 / speed_multiplier;  // 1000x speed = 1ms sleep

    while (!g_shutdown) {
        // Update all simulation Pieces
        update_weather_pieces();
        update_geology_pieces();
        update_economy_pieces();

        // Log prediction state
        log_prediction_step();

        usleep(sleep_ms * 1000);
    }
}

void update_weather_pieces() {
    // Read current weather state
    int temp = get_state_int("earth_atmosphere", "temperature");
    int humidity = get_state_int("earth_atmosphere", "humidity");
    int pressure = get_state_int("earth_atmosphere", "pressure");

    // Simple weather model
    if (humidity > 80 && pressure < 1000) {
        // Rain likely
        set_state_int("earth_surface", "precipitation", 1);
        humidity -= 10;
    } else {
        set_state_int("earth_surface", "precipitation", 0);
    }

    // Update temperature based on time of day
    time_t now = time(NULL);
    struct tm *tm = localtime(&now);
    int hour = tm->tm_hour;
    int base_temp = 15 + 10 * sin((hour - 6) * 0.2618);  // Peak at noon
    set_state_int("earth_atmosphere", "temperature", base_temp);

    hit_frame_marker();
}
```

[VIEW 戏] The Theater of Sovereign Truth

Whether it's a heartbeat, a stock trade, or a supernova, it all follows the True Piece Method.

1. PIECE: The entity (Atom, Person, Planet).
2. MIRROR: The current state (Charge, Wealth, Temperature).
3. PULSE: The laws of physics/economics updating the reality.

"The files are the light; the simulation is the mirror of the universe." [* 星][COSMOS 宇]

[WARN 警] Common Pitfalls

Pitfall 1: Simulation Running Too Fast

Symptom: State files become corrupted or inconsistent.

Cause: Multiple updates happening before filesystem flushes complete.

Fix: Add minimum sleep between simulation steps:

```
usleep(10000);  // Minimum 10ms between steps
```

Pitfall 2: State Dependency Cycles

Symptom: Simulation oscillates or produces nonsensical results.

Cause: Piece A depends on Piece B which depends on Piece A (circular dependency).

Fix: Detect and break cycles in dependency graph:

```
// Topological sort before simulation step
topological_sort(all_pieces);
for (int i = 0; i < piece_count; i++) {
    update_piece(sorted_pieces[i]);
}
```

Pitfall 3: Memory Exhaustion from Large Simulations

Symptom: System slows to a crawl or crashes.

Cause: Too many Pieces being updated simultaneously.

Fix: Chunk simulation updates:

```
// Update 100 pieces per frame
int batch_size = 100;
for (int i = 0; i < total_pieces; i += batch_size) {
    update_piece_batch(&pieces[i], batch_size);
    usleep(1000);  // Brief pause between batches
}
```

[BLD 馆] Scholar's Corner: The "Day the Earth Stood Still"

One evening, a developer set the "Earth Sim" (Predictor) to run at 100,000x real-time speed. For hours, the system processed geological shifts and climate models. Suddenly, the entire OS seemed to freeze. Panicked, the developer checked the `ledger.txt` and found a single line: "REACHED_END_OF_SIMULATION: Sun has expanded to Earth's orbit." The system hadn't frozen; it had simply finished its task of simulating the next five billion years of the solar system. This was the moment we truly understood that TPMOS could simulate any timeline. [EARTH 地][星]

[DOC 记] Study Questions

1. In TPMOS, how is a cell Piece different from an organ Piece?
2. How does the "Predictor" use the Pulse (Chapter 3) to simulate future events?
3. How can TPMOS be used as a "Molecular Prototyping Lab" for drug discovery?
4. Write the code for a simple "population growth" simulation Piece.
5. True or False: A marketing predictor in TPMOS is a PAL script attempting to change a bot's `state.desire`.
6. Scenario: Your weather simulation produces unrealistic results (temperature jumps from -50°C to 100°C in one step). What's the likely cause?

[Return to Index](INDEX.md)

[BIZ 业] Chapter 13: Piecemark Labs & The Sovereign Venture

TPMOS isn't just an OS—it's a Sovereign Venture. Piecemark Labs operates in stealth, building the infrastructure that will power the next generation of decentralized applications. [CORP 司]

The Stealth Strategy

Piecemark Labs follows a Stealth-First approach:

- **Build in Silence:** Core infrastructure is built and tested internally before public release.
- **SSI Guard:** Self-Sovereign Identity (SSI) is the foundation. Every user owns their data, their keys, their destiny.
- **Infrastructure Moat:** The real value isn't the apps—it's the filesystem-as-database, the Piece architecture, and the PAL orchestration layer.

[$ 钱] Monetization Models

TPMOS supports multiple business models through its Piece-based architecture:

B2B (Business-to-Business)

- **Enterprise TPMOS:** Companies license TPMOS for internal operations (inventory, HR, project management).
- **Custom Ops Development:** Build industry-specific ops for clients.

B2C (Business-to-Consumer)

- **Premium Apps:** Sell specialized apps (GL-OS games, AI-Labs integrations).
- **Dividend Points:** Users earn Div-Points for participation, exchangeable for premium content.

B2G (Business-to-Government)

- **Civic Infrastructure:** TPMOS as a platform for transparent government services.
- **Auditability:** All state is plaintext, fully auditable by design.

B2E (Business-to-Employee)

- **Internal Tooling:** Companies build custom tools for employees using TPMOS app factory.
- **Onboarding Automation:** PAL scripts automate employee onboarding workflows.

The Onboarding Loop

New users are onboarded through automated PAL scripts:

Code Example: Onboarding Bot PAL Script

```
; onboarding_new_user.asm
start:
    ; Create user profile
    OP user::create_profile "new_employee"
    sleep 200000

    ; Assign default permissions
    OP user::assign_role "new_employee" "standard_user"
    sleep 100000

    ; Create workspace
    OP playrm::create_piece "workspace" "new_employee_desk"
    sleep 100000

    ; Send welcome message
    OP p2p::compose_message "new_employee" "admin" "Welcome" "Welcome to TPMOS!"
    sleep 100000

    ; Log completion
    call bot_tester::log_pass "Onboarding complete"
    hit_frame
    halt
```

Direct Marketing Contracts

In TPMOS, "Like = Contract". When a user likes a project or creator, a PAL-based contract is formed:

Code Example: Smart Contract Template

```
; direct_marketing_contract.asm
start:
    ; Read the "like" action
    read_state r1, "action_piece", "type"
    beq r1, r0, no_like

    ; Verify both parties exist
    call user::create_profile "creator"
    call user::create_profile "supporter"

    ; Form contract: supporter gets updates, creator gets engagement
    OP p2p::escrow_create "contract_like_001" "supporter" "creator"
    sleep 100000

    ; Set contract terms
    set_state_int("contract_like_001", "type", 1)  ; 1 = marketing contract
    set_state_int("contract_like_001", "duration", 365)  ; 365 days
    hit_frame

    j contract_formed

no_like:
    halt

contract_formed:
    call bot_tester::log_pass "Marketing contract formed"
    halt
```

[VAL 宝] Token/Div-Points Data Structure

Dividend Points are stored in Piece state:

```
[DIVIDEND_POINTS]
holder=wallet_001
balance=1500
last_claim=1773622576
earned_from=storage,compute,participation
```

[BLD 馆] Business/Dev Bridge

How business logic maps to code:

Business Concept	TPMOS Implementation
"Like = Contract"	PAL script forming escrow between two wallets
"Rent storage"	Piece with `rental` state, `capacity`, `price_per_gb`
"Sell compute"	Piece with `compute_capacity` state, `cpu_cores`, `price_per_hour`
"User onboarding"	PAL script calling `user::create_profile`, `assign_role`, etc.
"Marketing campaign"	PAL script targeting bot Pieces with `state.desire` updates

---

[LAW 法] Legal Roadmap

- **Phase 1:** Internal contracts (PAL scripts between company wallets)
- **Phase 2:** Public contracts (open marketplace for contract formation)
- **Phase 3:** Legal recognition (explore jurisdictional compliance)

[WARN 警] Crypto Cautions

- **Regulatory Uncertainty:** Token models may face legal challenges in different jurisdictions.
- **Security Risks:** Private keys must be protected. Never store plaintext keys in production.
- **Scalability:** File-based blockchain has limits. Consider Merkle tree optimization for large chains.

[WARN 警] Common Pitfalls

Pitfall 1: Hardcoded Business Logic

Symptom: Changing contract terms requires code recompilation.

Cause: Contract logic hardcoded in C instead of parameterized in PAL.

Fix: Store contract terms in state files, read them at runtime:

```
int duration = get_state_int("contract", "duration");
int type = get_state_int("contract", "type");
```

Pitfall 2: Onboarding Script Failures

Symptom: New users missing permissions or workspace.

Cause: PAL script fails silently on one step, continues to next.

Fix: Add error checking after each OP call:

```
OP user::create_profile "new_employee"
sleep 200000
read_state r1, "new_employee", "name"
beq r1, r0, onboarding_failed
```

Pitfall 3: Div-Point Exploitation

Symptom: Users earning infinite Div-Points.

Cause: No rate limiting on point-earning actions.

Fix: Track last claim time and enforce cooldown:

```
time_t last_claim = get_state_int("wallet", "last_claim");
time_t now = time(NULL);
if (now - last_claim < 3600) return;  // 1 hour cooldown
```

[BLD 馆] Scholar's Corner: The "First Contract"

The first real-world contract formed on TPMOS wasn't between two humans—it was between two bots. A testing bot (Bot-33, from Chapter 9) "liked" a marketing bot's campaign, forming a PAL-based contract. The marketing bot automatically started sending updates to

Bot-33, and Bot-33 started earning Div-Points for engagement. The developers watched in amazement as two autonomous agents formed a legally-binding (in TPMOS law) agreement without human intervention. This was the moment we realized: "The bots are the customers too." [BOT 机][BIZ 业]

[DOC 记] Study Questions

1. Explain the "Like = Contract" concept in TPMOS.
2. How does the onboarding PAL script automate employee setup?
3. What are the four monetization models supported by TPMOS?
4. Write a PAL script that forms a contract between a creator and supporter.
5. Scenario: A user claims they earned 10,000 Div-Points in one hour. How would you investigate and fix this?
6. True or False: All business logic in TPMOS must be written in C code.

[Return to Index](INDEX.md)

[PEN 笔] Chapter 12: The Soul Pen & The Multiverse

TPMOS is the underlying engine for one of the most ambitious cosmic franchises in history: The Soul Pen. [COSMOS 宇][B 书][闪]

[PEN 笔] What is the Soul Pen?

In the world of Sol, the Soul Pen is a mystical 3D-printing wand powered by clone souls. It is the ultimate tool of creation, allowing users to shape reality itself.

- **The Centroid:** At **Centroid University**, students (centuries old but youthful in appearance) use the Soul Pen and **Tear-It Cards** to craft "pure creations" in a multiverse centered around the Golden Ratio. [VIEW 戏][SEA 鱼]
- **The Artifact:** Within TPMOS, the Soul Pen is represented as a high-leverage creation tool in `op-ed` (Chapter 5), bridging the gap between fiction and function.

A Multi-Genre Franchise

The Soul Pen universe spans 12 epic books and a massive array of AI-generated media:

- **Video Games:** MMORPGs like *Avanta.C* and survival sims like *GEM&I*.
- **Tabletop:** Card and board games managed via the **Tear-It Deck** system.
- **Lore:** The **DUSTOPEDIA**, a grand interdimensional library holding the "Impossible Books" of the universe. [B 阅][COSMOS 宇]
- **Community:** The **Underground Xircus**, an online store and community hub for all things Soul Pen.

[LAUNCH 发] The TPMOS Synergy

TPMOS provides the infrastructure that makes the Soul Pen's multiversal scale possible:

1. Sovereign Storage: Using the P2P-NET (Chapter 8), Soul Pen assets (comics, music, 3D models) are stored as sovereign Pieces.
2. Recursive Forge: The Soul Pen's "Magic Tree" and crafting systems are powered by the

same recursive logic used in the OS's chip design suites (Chapter 9). [REC 归]
3. AI Orchestration: AI-Labs (Chapter 8) acts as the "Ghost in the Machine," generating the vast, sprawling narratives of the 12-book cycle.

[VIEW 戏] The Soul Pen Roadmap

- **Phase 1:** Core infrastructure (TPMOS v03.00) [OK 是]
- **Phase 2:** GL-OS integration for 3D Soul Pen rendering (Chapter 8)
- **Phase 3:** P2P-NET for decentralized asset distribution (Chapter 10)
- **Phase 4:** AI-Labs for narrative generation (Chapter 10)
- **Phase 5:** Full multiverse launch

[WARN 警] Common Pitfalls

Pitfall 1: Scope Creep

Symptom: Project grows beyond manageable size, development stalls.
Cause: Trying to build the entire multiverse at once.
Fix: Focus on one Piece at a time. Build the Soul Pen as a single Piece first, then expand.

Pitfall 2: Fiction/Function Confusion

Symptom: Developers confuse lore features with actual implemented features.
Cause: Blurring the line between narrative and code.
Fix: Clearly document which features are implemented in code vs. planned for future. Use `2do.txt` for planned features.

[BLD 馆] Scholar's Corner: The "Impossible Book"

The DUSTOPEDIA contains "Impossible Books"—volumes so vast they cannot be fully read by

any single mind. In TPMOS, these are represented as recursive Piece structures that generate infinite content through PAL scripts. The first Impossible Book was a 4-billion-page novel generated by an AI-Labs script that ran for three days. It was never fully read, but fragments of it became the basis for entire game worlds. This taught us that "Some stories are too big for one lifetime." [B 阅][INF 无]

[DOC 记] Study Questions

1. How does TPMOS provide the infrastructure for the Soul Pen multiverse?
2. What is the relationship between the Soul Pen and `op-ed`?
3. Imagine: You want to create a Soul Pen card game using TPMOS. What Pieces would you need?
4. Critical Thinking: How does the "Impossible Book" concept relate to recursive chip design (Chapter 11)?

[Return to Index](INDEX.md)

- **Sol Pen (Book 1):** Lucky Shepherd's journey from casino mogul to lunar savior. [LUCK 赌][COSMOS 宇]
- **Centroid University:** Lucky and Astra's magical education in the multiverse. [* 星][B 书]
- **Superwar Ikkarus:** The battle against the Metal Sun and the Architect's cosmic tyranny. [LAUNCH 发]

"The bot writes its own ops. We just light the fuse." [FIRE 炮] In the Soul Pen, the user writes the reality; TPMOS simply hosts the dream.

[Return to Index](INDEX.md)

Chapter 15: Cross-Platform TPMOS (Windows/Linux/macOS)

TPMOS is designed from the ground up to run on any platform. This chapter covers the differences, patterns, and setup guides for Windows, Linux, and macOS. [CPU 算]

[BLD 建] Build Systems

Linux

```
# Compile all binaries
./#.tools/compile_all.sh

# Run CHTPM
./button.sh run

# Kill all processes
./button.sh kill
```

Windows (MSYS2/MinGW)

```
# Compile all binaries
.\pieces\buttons\windows\legacy\compile_all.psl

# Run CHTPM
.\pieces\buttons\windows\run.psl

# Kill all processes
.\pieces\buttons\windows\kill.psl
```

macOS

```
# Same as Linux (bash scripts work on macOS)
./#.tools/compile_all.sh
./button.sh run
```

Platform-Specific Code

Input Handling

Platform	File	API
Linux	`pieces/keyboard/src/keyboard_input_linux.c`	`termios`, raw terminal
Windows	`pieces/keyboard/src/keyboard_input_win.c`	`GetAsyncKeyState()`, `ReadConsoleInput()`
macOS	`pieces/keyboard/src/keyboard_input_linux.c`	Same as Linux (termios works)

Rendering

Platform	File	API
Linux	`pieces/display/renderer.c`	Terminal ASCII
Windows	`pieces/display/windows_renderer.c`	Windows Console API
macOS	`pieces/display/renderer.c`	Terminal ASCII

Process Spawning

Platform	Pattern
Linux/macOS	`fork()` / `exec()` / `waitpid()`
Windows	`CreateProcess()` / `WaitForSingleObject()`

Cross-Platform Pattern

```
#ifdef _WIN32
    #include <windows.h>
    #include <direct.h>
    #include <io.h>
    #define usleep(us) Sleep((us)/1000)
    #define access _access
    #define F_OK 0
    #define getcwd _getcwd
#else
    #include <unistd.h>
    #include <sys/stat.h>
    #include <sys/wait.h>
#endif
```

Path Differences

Aspect	Linux/macOS	Windows
Path separator	`/`	`\`
Binary extension	`.+x`	`.exe`
Home directory	`/home/user/`	`C:\Users\user\`
Font path	`/usr/share/fonts/`	`C:\Windows\Fonts\`

Solution: Use `build_path_malloc()` Helper

```
char* build_path_malloc(const char* rel) {
    size_t sz = strlen(project_root) + strlen(rel) + 2;
    char* p = (char*)malloc(sz);
    if (p) snprintf(p, sz, "%s/%s", project_root, rel);
    return p;
}
```

Fondu on Windows

PowerShell Installer vs Bash Script

- Linux/macOS: `./fondu --install <project>` (bash)
- Windows: `.\fondu.ps1 -Install <project>` (PowerShell)

Binary Convention

- Linux/macOS: `.+x` extension
- Windows: `.exe` extension

The compile scripts handle this automatically. Fondu looks for the correct extension based on platform.

[LAB 验] Testing Across Platforms

Cross-Platform Ops

Most ops are cross-platform because they only do file I/O:

- `move_entity.+x` [OK 是]
- `render_map.+x` [OK 是]
- `create_profile.+x` [OK 是]
- `auth_user.+x` [OK 是]

Platform-Specific Ops

These ops need platform-specific versions:

- `keyboard_input.+x` [FAIL 错] (different per platform)
- `renderer.+x` [FAIL 错] (different per platform)
- `joystick_input.+x` [FAIL 错] (XInput on Windows, evdev on Linux)

CI/CD Considerations

- GitHub Actions: Run tests on Linux, Windows, macOS runners
- Compile verification: Ensure all ops compile on all platforms
- Integration tests: Run full test suite on each platform

[TOOL 工] Developer Setup Guides

Linux (Already Working)

Your current setup is Linux. You're good to go!

Windows (MSYS2)

1. Install MSYS2: Download from https://www.msys2.org/
2. Install GCC: `pacman -S mingw-w64-x86_64-gcc`
3. Clone TPMOS: `git clone <repo>`

4. Compile: `.\pieces\buttons\windows\legacy\compile_all.psl`
5. Run: `.\button.psl run`

macOS

1. Install Xcode Command Line Tools: `xcode-select --install`
2. Install Homebrew: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
3. Install dependencies: `brew install glfw freetype`
4. Compile: `./#.tools/compile_all.sh`
5. Run: `./button.sh run`

[WARN 警] Common Pitfalls

Pitfall 1: Hardcoded Linux Paths on Windows

Symptom: "File not found" errors on Windows.

Cause: Paths use `/` but Windows expects `\`.

Fix: Use `build_path_malloc()` or `snprintf()` with platform-aware path construction.

Pitfall 2: Missing MSYS2 Dependencies on Windows

Symptom: Compile fails with "undefined reference to `glfwInit`".

Cause: GLFW not installed in MSYS2.

Fix: `pacman -S mingw-w64-x86_64-glfw`

Pitfall 3: Signal Handling Differences

Symptom: Ctrl+C doesn't clean up children on Windows.

Cause: Windows signal handling differs from POSIX.

Fix: Use `SetConsoleCtrlHandler()` on Windows:

```
#ifdef _WIN32
BOOL WINAPI ctrl_handler(DWORD type) {
    if (type == CTRL_C_EVENT) {
        g_shutdown = 1;
        return TRUE;
    }
    return FALSE;
}
SetConsoleCtrlHandler(ctrl_handler, TRUE);
#else
signal(SIGINT, handle_sigint);
#endif
```

Pitfall 4: Font Path Not Found on GL-OS

Symptom: GL-OS crashes with "Failed to load font".

Cause: Font path hardcoded to Linux location.

Fix: Use platform-specific font paths:

```
#ifdef _WIN32
    const char* font_path = "C:/Windows/Fonts/arial.ttf";
#elif __APPLE__
    const char* font_path = "/System/Library/Fonts/Supplemental/Arial.ttf";
#else
    const char* font_path = "/usr/share/fonts/truetype/arphic/uming.ttc";
#endif
```

[BLD 馆] Scholar's Corner: The "Great Migration"

When TPMOS was first ported to Windows, the developer assumed it would be a simple matter of changing `#include` headers. Instead, they discovered that every single path in the codebase was hardcoded with `/home/user/`. It took three weeks to refactor every file to use `build_path_malloc()`. But when the first Windows build finally compiled and ran, the developer watched in awe as the ASCII terminal rendered perfectly, the zombie chased the player, and the entire TPMOS universe existed natively on Windows. This became known as "The Great Migration." It taught us that "Portability is not an afterthought—it is a design principle." [EARTH 地]

[DOC 记] Study Questions

1. What is the Windows equivalent of `fork()/exec()`?
2. Why does TPMOS use `.+x` on Linux but `.exe` on Windows?
3. Write the code to make signal handling work on both Windows and Linux.
4. Scenario: Your op compiles on Linux but fails on Windows with "undefined reference to `usleep`". What's the fix?
5. True or False: All TPMOS ops are cross-platform by default.

[Return to Index](INDEX.md)

Chapter 16: Common Pitfalls & Debugging Guide

This chapter catalogs every known TPMOS pitfall and provides a comprehensive debugging workflow. Keep this as your reference when things go wrong. [TOOL 工]

Quick Diagnostic Flowchart

```
Problem → Is it a compile error?
  YES → Check CH4 (Muscles & Brains) for compile patterns
  NO → Is it a runtime crash?
    YES → Check signal handling (CH4), file paths (CH2)
    NO → Is it a logic bug (wrong behavior)?
      YES → Check state files, PAL scripts, pipeline flow
      NO → Is it a performance issue?
        YES → Check CPU-safe patterns (CH4), stat() guard (CH2)
```

[DOC 单] Pitfall Index by Chapter

CH2: Filesystem

#	Pitfall	Symptom	Fix
1	Hardcoded paths	Works on your machine, fails after moving	Use `resolve_paths()`
2	Forgetting `location_kvp` update	Entire project "disappears"	Update `project_root=` line
3	`stat()` race conditions	Input missed or processed twice	Use `fseek()` + `ftell()` pattern
4	Reading state without checking	Crash or garbage values	Check `fopen()` return value

CH3: Pipeline

#	Pitfall	Symptom	Fix
1	Using `system()`	Orphaned processes after Ctrl+C	Use `fork()/exec()/waitpid()`
2	Forgetting `waitpid()`	Zombie processes accumulate	Always call `waitpid()`
3	Not hitting `frame_changed.txt`	Screen doesn't refresh	Call `hit_frame_marker()`
4	Blocking pipeline with slow ops	System stutters	Keep ops under 16ms

CH4: Modules & Ops

#	Pitfall	Symptom	Fix
1	Orphaned processes on Ctrl+C	Game keeps running after quit	Register `handle_sigint`, use `setpgid(0,0)`
2	Tight loop without sleep	100% CPU when idle	Always `usleep(16667)` or `usleep(100000)`
3	Scattering ops	Fondu can't find ops	Centralize in `ops/` directory
4	Forgetting `ops_manifest.txt`	Fondu skips project	Create manifest with correct format
5	Thick brain anti-pattern	5000+ line manager	Move logic into separate ops

CH6: PAL

#	Pitfall	Symptom	Fix
1	Infinite loops	System hangs, CPU spikes	Ensure loops have terminating condition
2	Forgetting `sleep` between ops	Ops fail silently	Add `sleep 50-200` between heavy ops
3	Cross-project call failures	`OP other::op` does nothing	Install target project via Fondu first
4	Not ending with `halt`	Interpreter crashes	Every path must end with `halt`

CH7: fuzz-op & op-ed

#	Pitfall	Symptom	Fix
1	Zombie AI not moving	Zombie stays in place	Ensure `zombie_ai.+x` exists and uses `resolve_paths()`
2	Manager running from wrong directory	Nothing works	Kill, recompile from correct dir, restart
3	Selector state corruption	Selecting entity breaks movement	Update selector's `map_id` to match entity
4	op-ed changes not appearing	Map doesn't update	Ensure `hit_frame_marker()` after tile placement
5	Z-level desync	Map doesn't change on Z move	Update `current_z_val`, write correct map name

CH8: GL-OS

#	Pitfall	Symptom	Fix
1	Font file not found	GL-OS crashes on startup	Use dynamic font path resolution
2	ASCII/GL desync	3D doesn't match terminal	Ensure both read same frame file
3	Model path resolution	Models appear as blank boxes	Use absolute paths from `project_root`
4	OpenGL context not created	Black/blank window	Check `glfwCreateWindow()` return value

CH9: Testing

#	Pitfall	Symptom	Fix
1	Bot moving too fast	Keys not processed	Add `sleep` between injections
2	Assertion timing	Test fails intermittently	Poll until state stabilizes
3	Bot state corruption	Wrong state between runs	Reset bot state before each test

|

CH10: P2P-NET

#	Pitfall	Symptom	Fix
1	Ghost port listeners	"Address already in use"	Kill processes, verify port free
2	Blockchain file corruption	Chain validation fails	Use file locking for writes
3	Wallet state mismatch	Shows connected but no activity	Update state on connect AND disconnect
4	Cross-project PAL failures	`OP p2p::op` does nothing	`./fondu --install p2p-net` first

CH11: Recursive Forge

#	Pitfall	Symptom	Fix
1	Gate evaluation order	Wrong circuit output	Topologically sort gates
2	Clock too fast	State doesn't propagate	Minimum 50ms clock period
3	Infinite PAL recursion	Stack overflow	Ensure recursive `call` has base case

CH12: Simulation

#	Pitfall	Symptom	Fix
1	Simulation too fast	State corrupted	Minimum 10ms between steps
2	State dependency cycles	Nonsensical results	Detect and break cycles
3	Memory exhaustion	System slows/crashes	Chunk simulation updates

CH13: Business

#	Pitfall	Symptom	Fix

1	Hardcoded business logic	Requires recompile for term changes	Store terms in state files
2	Onboarding script failures	Missing permissions	Add error checking after each OP
3	Div-Point exploitation	Infinite points earned	Track last claim time, enforce cooldown

CH14: Soul Pen

#	Pitfall	Symptom	Fix
1	Scope creep	Development stalls	Focus on one Piece at a time
2	Fiction/function confusion	Lore vs code blur	Document implemented vs planned

CH15: Cross-Platform

#	Pitfall	Symptom	Fix
1	Hardcoded Linux paths on Windows	"File not found"	Use `build_path_malloc()`
2	Missing MSYS2 dependencies	Compile fails	Install required packages
3	Signal handling differences	Ctrl+C doesn't cleanup	Use `SetConsoleCtrlHandler()` on Windows
4	Font path not found	GL-OS crashes	Use platform-specific font paths

---

Debugging Workflow

Step 1: Identify the Symptom

What exactly is happening?

- [ ] Compile error? → Check error message, verify file exists
- [ ] Runtime crash? → Check signal handling, file paths
- [ ] Wrong behavior? → Check state files, PAL scripts
- [ ] Performance issue? → Check CPU-safe patterns

Step 2: Check the Debug Log

```
cat projects/fuzz-op/manager/debug_log.txt | tail -20
```

Look for:

- Last successful operation
- Error messages
- State changes

Step 3: Verify State Files

```
# Check piece state
cat projects/fuzz-op/pieces/world_01/map_01/zombie_01/state.txt

# Check manager state
cat pieces/apps/player_app/manager/state.txt

# Check location_kvp
cat pieces/locations/location_kvp
```

Step 4: Verify Binaries Exist

```
# Check if binary exists
ls -la pieces/apps/fuzzpet_app/traits/+x/zombie_ai.+x

# Check if it's executable
file pieces/apps/fuzzpet_app/traits/+x/zombie_ai.+x
```

Step 5: Check Process List

```
# See what's running
ps aux | grep -E "(fuzz-op|chtpm|manager)" | grep -v grep

# Kill everything
./button.sh kill
```

Step 6: Recompile

```
./button.sh compile
./button.sh check
```

Step 7: Restart

```
./button.sh run
```

[TOOL 工] Common Debug Commands

View Recent Debug Output

```
tail -50 projects/fuzz-op/manager/debug_log.txt
```

Check All Binaries

```
./button.sh check
```

Kill All Processes

```
./button.sh kill
```

Check Port Usage (P2P issues)

```
lsof -i :8000
```

Verify File Permissions

```
ls -la pieces/apps/fuzzpet_app/traits/+x/
```

Test Single Op

```
./pieces/apps/playrm/ops/+x/move_entity.+x selector up
```

Health Check Script

Create a health check script to verify system state:

```
#!/bin/bash
# health_check.sh - Verify TPMOS system health

echo "=== TPMOS Health Check ==="

# Check location_kvp
if [ -f "pieces/locations/location_kvp" ]; then
    echo " location_kvp exists"
else
    echo " location_kvp MISSING"
fi

# Check binaries
BINARIES=(
    "pieces/chtpm/plugins/+x/chtpm_parser.+x"
    "pieces/apps/playrm/ops/+x/move_entity.+x"
    "pieces/apps/fuzzpet_app/traits/+x/zombie_ai.+x"
)

for bin in "${BINARIES[@]}"; do
    if [ -f "$bin" ]; then
        echo " $bin"
    else
        echo " $bin MISSING"
    fi
done

# Check state files
if [ -f "pieces/apps/player_app/manager/state.txt" ]; then
    echo " Manager state exists"
else
    echo " Manager state MISSING"
fi

# Check for orphaned processes
ORPHANS=$(ps aux | grep -E "(manager|parser|renderer)" | grep -v grep | wc -l)
if [ $ORPHANS -gt 0 ]; then
    echo "[WARN 警] $ORPHANS processes running"
else
    echo " No orphaned processes"
fi

echo "=== Health Check Complete ==="
```

[BLD 馆] Scholar's Corner: The "Debug That Debugged Itself"

A developer once wrote a PAL script to automatically diagnose and fix common TPMOS issues. The script would check state files, verify binaries, restart processes, and even recompile missing ops. One day, the script encountered an error it couldn't fix: its own source file had been corrupted. Instead of failing, the script regenerated itself from the compile script and continued running. This became known as "The Debug That Debugged Itself." It taught us that "The best debugger is one that can fix itself." [INF 无]

[DOC 记] Study Questions

1. You press Ctrl+C but the game keeps running. What are three possible causes?
2. The zombie isn't moving. Walk through the diagnostic steps from this chapter.
3. Scenario: Your PAL script hangs after calling an OP. What's the most likely cause?
4. Write a health check that verifies all P2P-NET binaries exist.
5. True or False: The first step in debugging is always to recompile.
6. Critical Thinking: Why is checking `debug_log.txt` more useful than just restarting the system?

[Return to Index](INDEX.md)

[B 阅] Glossary of Terms

Welcome to the official TPMOS Lexicon. This glossary defines the core concepts and the PJARGO (Internal Dialect) used within the ecosystem. [BLD 馆]

[BLD 馆] Core Concepts

- **TPM (True Piece Method):** The governing philosophy where reality is decomposed into atomic "Pieces" that own their own state.
- **PMO (Piece-Module-OS):** The 3-tier architectural hierarchy for system development.
- **Piece:** A directory containing a `.pdl` (DNA) and `state.txt` (Mirror). The "Soul" of an object.
- **Module:** A background C-process ("Brain") that manages Pieces and handles input.
- **Mirror:** The `state.txt` file representing a Piece's current physical/logic state.
- **DNA:** The `.pdl` file defining a Piece's potential, methods, and traits.
- **Pulse:** The recurring heartbeat of the 12-step pipeline that updates the frame.
- **Sovereignty:** The principle that a Piece exclusively owns its files; no other process should overwrite its state directly.

PJARGO (Internal Dialect)

- **W-first:** A world-first path resolution strategy (`world_01/map_01/...`).
- **RT Parity:** Roundtrip parity—the guarantee that Saving and Loading results in bit-for-bit identical state.
- **Ghost Hardcode:** A forbidden practice where project-specific assumptions are hidden in generic system code. [GHOST 鬼]
- **Muscle Layer:** The "Ops" binaries (`.+x`) that perform the actual work for a Brain Module.

- **Sea Shell:** The core system (Parser, Nav) is strong; growth occurs at the flexible edges (Apps/Projects).
- **Mirror Sync:** The process of flushing in-memory changes to the `state.txt` file.
- **W-first (World-First):** The canonical path resolution strategy that prioritizes `world_<id>/map_<id>/` before checking legacy fallback paths. [MAP 图]
- **Mode 1 (Realtime):** The direct C-based execution path for high-speed input like movement (latency ~16ms).
- **Mode 2 (Event):** The PAL-based execution path for complex scripts, AI, and dialogue (latency ~50-100ms).
- **Thin Brain:** A module that delegates logic to ops rather than implementing it directly (opposite of "Thick Brain").
- **Thick Brain:** An anti-pattern where a module implements all logic internally, leading to bloat and instability.

[TOOL 齿] Technical Terms

- **PAL (Prisc Assembly Language):** The RISC-like scripting language used for AI and events.
- **prisc+x:** The PAL interpreter binary.
- **KVP (Key-Value Pair):** The primary data format used in `.pdl`, `state.txt`, and `location_kvp`.
- **CHTPM:** The "CH" range of TPM, typically referring to the character-based (ASCII) OS shell.
- **GL-OS:** The OpenGL visual theater that mirrors the CHTPM shell.
- **.+x:** TPMOS binary extension on Linux/macOS; `.exe` on Windows.
- **CreateProcess():** Windows equivalent of `fork()/exec()`.
- **Fondu:** Package manager for TPMOS ops and apps.
- **P2P-NET:** Decentralized networking layer.
- **Div-Points:** Dividend Points, earned through participation in the network.
- **Escrow:** File-based contract enforcement mechanism.
- **Block:** Unit of blockchain ledger.
- **Wallet Piece:** P2P identity container.

- **Frame Bridge:** Sync mechanism between ASCII and GL rendering.
- **op-ed:** TPMOS piece editor application.
- **fuzz-op:** TPMOS pet simulation project.
- **stat()-first:** CPU-safe polling pattern that only reads files when size changes.
- **fork()/exec()/waitpid():** POSIX process spawning pattern for CPU-safe op execution.
- **setpgid():** Sets process group for clean Ctrl+C cleanup.
- **hit_frame_marker():** Function that triggers render pipeline by writing to `frame_changed.txt`.
- **resolve_paths():** Function that reads `location_kvp` to determine project root.
- **build_path_malloc():** Helper that constructs platform-aware paths.

[Return to Index](INDEX.md)

[DOC 记] The TPMOS Final Exam (Q&A)

Congratulations, Scholar! You have reached the end of the TPMOS Textbook. It is time to test your knowledge of the Mono-OS. [GRAD 学][LAUNCH 发]

[BLD 馆] Part I: Foundational Philosophy

Q1: What are the three layers of the PMO hierarchy?

- *A: PIECE (The Atom), MODULE (The Brain), and OS (The Theater).*

Q2: What is the core mandate of the True Piece Method (TPM)?

- *A: "If it's not in a file, it's a lie." Every piece of reality must be mirrored in a state file.*

Q3: Can a galaxy and a button both be considered "Pieces"?

- *A: Yes! TPMOS is scale-free. The same rules apply to a single pixel and a billion stars.*

[TOOL 齿] Part II: Machinery & Pipeline

Q4: How many steps are in the TPMOS operational pulse?

- *A: 12 steps, from Input to Display.*

Q5: Why do we use `fork()` and `exec()` instead of the `system()` command?

- *A: For CPU safety and process control. `system()` is hard to kill; `fork()` allows us to manage the child's life cycle.*

Q6: What is the "Stat-First Pattern"?

- *A: Only opening a file if its modification time (`mtime`) has changed. This keeps CPU usage at 0% when the system is idle.*

[LAUNCH 发] Part III: Advanced Simulation

Q7: What is a "Testing Bot" in TPMOS?

- *A: A sovereign Piece that uses PAL scripts to "play" the OS like a human user.*

Q8: How does LSR use AI-Labs for R&D?

- *A: AI-Labs distills natural language goals into actual game tech, recipes, and Piece DNA in real-time.*

Q9: What is "Recursive Chip Design"?

- *A: Building logic gates (NAND, NOR) as Pieces on a map, allowing the OS to simulate its own future hardware.*

Q10: Can TPMOS simulate the five-billion-year future of the Sun?

- *A: Yes! By running the Pulse at high speed, the Simulation Theater can predict any timeline.*

Graduation

If you answered these correctly, you are officially a Certified TPMOS Architect. Go forth and build the future! [星][COSMOS 宇]

[Return to Index](INDEX.md)

Answer Key: Study Questions

Mastered the Mono-OS? Check your answers here! [GRAD 学][LAUNCH 发]

[DNA 命] Chapter 1: The Soul of a Piece

1. Layers of PMO: PIECE (Atom/State), MODULE (Brain/Logic), OS (Theater/View).
2. DNA vs. Mirror: DNA (.pdl) defines potential and methods; Mirror (state.txt) defines current runtime state.
3. No In-Memory HP: Memory is volatile ("a lie"). Files ensure reality persists even if a process crashes.
4. True or False: True. TPMOS is scale-free; both are sovereign Pieces.

Chapter 2: The Filesystem is the Database

1. location_kvp: Acts as the system's "Ground" or "Compass," defining absolute paths for the project root and pieces.
2. File-Based IPC: Programs communicate by writing notes (files) like `history.txt` or `pulse.txt` instead of sharing RAM.
3. Stat-First Pattern: It checks the file's modification time before opening it, keeping CPU at 0% when idle.
4. Moving a Project: Update the `project_root` line in `pieces/locations/location_kvp`.

[PULSE 心] Chapter 3: The 12-Step Pulse

1. Keystroke Journey: Input -> History -> Parser -> App Buffer -> Module Tick -> Op Exec -> Piece Sovereignty -> Mirror Sync -> Stage Producer -> Parser Composition -> Final Render -> Display.
2. Stage Producer: Reads all Mirrors on a map and draws the composite "View" (`view.txt`).
3. Mirror Sync Importance: Ensures reality is written to disk so the Stage Producer sees the latest state.
4. Stutter Culprit: Step 5 (Muscle Op execution) or Step 8 (Stage Production) taking

too long.

Chapter 4: Muscles & Brains

1. Module vs. Op: A Module is a long-running "Brain"; an Op is a short-lived, single-task "Muscle."
2. Fork/Exec vs. System: Fork/Exec gives full control over the child's lifecycle; `system()` is hard to track and kill.
3. CPU-Safe Mandate: Focus-aware throttling, Stat-first polling, and Signal handling (SIGINT).
4. True or False: True. It should sleep ~100ms when inactive.

[APP 厂] Chapter 5: The App Factory

1. App Requirement: A directory containing an app-id DNA file (`.pdl`) and a `layouts/` folder.
2. User Profiles: Managed by the `user` app.
3. Adding to Menu: Add a `<button href="...">` pointing to the app's `.pdl` in `os.chtpm`.
4. True or False: True. The `.pdl` is the descriptor the OS needs to launch the app.

[GL 艺] Chapter 6: Beyond ASCII (GL-OS)

1. OS or Visualizer: It is a high-fidelity visualizer shell that mirrors ASCII reality.
2. Mapping Models: It parses specific characters (like `[P]`) and maps them to 3D model paths in its configuration.
3. Shadow vs. Light: Pixels are just a visual representation (shadow) of the true data stored in files (light).
4. Update Color: Yes. If the character or its associated state changes, GL-OS updates the render instantly.

[BOT 机] Chapter 7: The Guardians (Testing)

1. Bot is a User: A bot uses the same input/output channels as a human, making tests "reality-accurate."

2. FSM States: IDLE, NAVIGATING, INTERACTING, ASSERTING.
3. Input Pipeline: It appends timestamped `KEY_PRESSED` lines to `keyboard/history.txt`.
4. P2P Test Script: A PAL script that spawns two bots to trade an NFT and asserts the balance shift.

[EXP 卫] Chapter 8: Future Horizons

1. AI-Labs in LSR: It generates dynamic tech recipes, item names, and `.pdl` stats based on R&D "actions."
2. Unified Contract Model: Allows the same legal/logic code to run on a blockchain (P2P) or in a world sim (LSR Gov).
3. Mining Coins: By providing disk storage or CPU compute cycles to the infrastructure network.
4. Compute Piece: You publish a "Worker Piece" with your CPU specs and an "Open" availability state.

[FAST 快] Chapter 9: The Infinite Loop

1. Recursive Chip Design: Building logic gates as Pieces so the OS can simulate and "test" its own hardware.
2. Scale-Free Simulation: Since a Piece can be anything, a NAND gate follows the same state-rules as a Hero.
3. In-Game Programming: Players use an in-game `op-ed` to write PAL scripts for their "Auto-Battler" bots.
4. True or False: False. Both follow the same PIECE -> MODULE -> OS hierarchy.

[LAB 验] Chapter 10: The Simulation Theater

1. Cell vs. Organ: A cell is a single Piece; an organ is a Container Piece holding millions of cell Pieces.
2. Predictor Pulse: It runs the 12-step loop much faster than 60 FPS to calculate far-future states.
3. Molecular Lab: Bots "play" chemistry by testing millions of molecular Piece

configurations for "Success" states.

4. True or False: True. Marketing is the attempt to program a bot's internal state.

[Return to Index](INDEX.md)

PIECEMARK-IT / CHTPM+OS Monolith

Version: 6.0 (Standardized Ops Edition)

Date: April 2, 2026

Philosophy: PIECE -> MODULE -> OS (PMO)

This is the central documentation for the Piecemark-IT ecosystem, a file-backed, auditable, and modular OS environment built on the True Piece Method (TPM).

Table of Contents

1. [Core Philosophy (PMO/TPM)](#1-core-philosophy-pmotpm)
2. [PJARGO Dictionary (Internal Dialect)](#2-pjargo-dictionary-internal-dialect)
3. [System Architecture](#3-system-architecture)
4. [App vs. Project (Critical Distinction)](#4-app-vs-project-critical-distinction)
5. [Fondu Lifecycle Manager](#5-fondu-lifecycle-manager)
6. [The Runtime Pipeline (12 Steps)](#6-the-runtime-pipeline-12-steps)
7. [Development Standards](#7-development-standards)
8. [How to Add to the Codebase](#8-how-to-add-to-the-codebase)
9. [Current Status & Roadmap](#9-current-status--roadmap)
10. [Tools & Maintenance](#10-tools--maintenance)

[DNA 命] 1. Core Philosophy (PMO/TPM)

The system follows a strict thought priority: PIECE -> MODULE -> OS.

- **PIECE (The Atomic Unit):** A directory containing text files defining state (`state.txt`) and DNA (`piece.pdl`). A piece owns its state exclusively. "If it's not in a file, it's a lie."
- **MODULE (The Brain):** Orchestrates pieces, interprets input, and executes **Ops**. It should delegate "Muscle" logic to standalone binaries.
- **OS / CHTPM (The Theater):** The environment and UI. It substitutes variables

(`${var}`), routes input, and composes the final frame. "The Parser is the mirror's eye, not the mirror's mind."

TPM Principles:

- **Data Sovereignty:** Each piece owns its files.
- **Auditability:** All major events log to `master_ledger.txt`.
- **File-Based IPC:** Communication happens via files, not memory.
- **Recursive Containment:** Maps contain pieces; pieces can contain pieces (inventory/interiors).

---

2. PJARGO Dictionary (Internal Dialect)

High-leverage vocabulary for developers and agents.

- **W-first:** World-first path resolution (`world_<id>/map_<id>/...`).
- **RT Parity:** Roundtrip parity (Save -> Load yields identical state).
- **Slice-Safe:** Isolate changes so other subsystems keep running.
- **Ghost Hardcode:** Hidden project-specific assumptions in generic code (AVOID).
- **Mirror:** The `state.txt` runtime representation.
- **DNA:** The `piece.pdl` definition layer.
- **Sea Shell:** The core (Parser, Nav) is strong; growth (Apps) is at the edges.

[BLD 建] 3. System Architecture

The filesystem is the database.

```
/
├── pieces/
│   ├── apps/          # System tools (op-ed, gl-os, user)
│   ├── chtpm/         # Parser, orchestrator, layouts
│   ├── display/       # current_frame.txt, current_layout.txt
│   ├── system/        # pdl_reader, kvp_db, prisc (VM)
│   └── world/         # Global world instances
├── projects/         # User-created content (fuzz-op, lsr, demo_l)
├── #.docs/           # Canonical documentation
├── #.plans/          # Future roadmap
└── #.tools/          # Maintenance scripts
```

World/Map/Piece Nesting Model:

`projects/<project>/pieces/world_<id>/map_<id>/<piece_id>/`

[LAUNCH 发] 4. App vs. Project (Critical Distinction)

Feature	APP (System Tool)	PROJECT (User Content)
Location	`pieces/apps/<app_id>/`	`projects/<project_id>/`
Launch	via `<button href="...">`	via Loader Menu or Fondu
Descriptor	`<app_id>.pdl` (layout_path)	`project.pdl` (entry_layout)
Editable	Yes (source in `pieces/apps/`)	Yes (source in `projects/`)
Production	N/A	Installed via Fondu to `pieces/apps/installed/`
Example	`player_app`, `chtpm` (system)	`op-ed`, `fuzz-op`, `user`, `test_fondu`

Note: Projects can be installed as production apps via Fondu (`./fondu --install <project>`).

---

5. Fondu Lifecycle Manager

Fondu is the TPMOS project lifecycle manager. It automates project deployment, ops registration, and archive management.

Location

`pieces/system/fondu/fondu.+x`

Commands

Command	Description
`--install <project>`	Compile, deploy to `pieces/apps/installed/`, register ops
`--uninstall <app>`	Remove from installed/, unregister ops
`--archive <project>`	Move to `projects/trunk/` (source only, not compiled)
`--restore <project>`	Move from trunk/ back to `projects/`, recompile
`--list`	Show all projects and their states
`--list-ops`	Show all available ops (from ops catalog)
`--help`	Show help message

Project States

```
┌──────────────────────────┐       ┌──────────────────────────┐       ┌──────────────────────────┐
│   ACTIVE     │       │   ARCHIVED   │       │   INSTALLED   │
│  projects/   │──────────> │ projects/trunk │──────────> │ pieces/apps/  │
│              │       │              │       │ installed/    │
│ - Editable   │       │ - Source     │       │ - Compiled    │
│ - In compile │       │ - Not built  │       │ - Read-only   │
│ - Dev-facing │       │ - Backup     │       │ - User-facing │
└──────────────────────────┘       └──────────────────────────┘       └──────────────────────────┘
      ↑                    ↑                    ↑
      └──────────────────────────────────────────┴──────────────────────────────────────────┘

                  Fondu moves projects between states
```

Ops Registry

When you install a project with ops, Fondu:

1. Registers ops in `pieces/os/ops_registry/<project>.txt`
2. Updates `pieces/os/compiled_projects.txt`
3. Generates `pieces/os/ops_catalog.txt` (human-readable)

Other projects can then call these ops via PAL scripts:

```
OP user::create_profile "player1"
OP user::auth_user "player1"
```

Example Workflow

```
# Install a project to production
./pieces/system/fondu/fondu.+x --install op-ed

# List all projects
./pieces/system/fondu/fondu.+x --list

# List available ops
./pieces/system/fondu/fondu.+x --list-ops

# Archive a project (move to trunk)
./pieces/system/fondu/fondu.+x --archive 1sr

# Restore from trunk
./pieces/system/fondu/fondu.+x --restore 1sr
```

Reference: `#.docs/fondu-investigation-report.txt` for complete documentation.

[TOOL 齿] 6. The Runtime Pipeline (12 Steps)

1. INPUT: `keyboard_input.+x` writes to `keyboard/history.txt`.
2. ROUTING: `chtpm_parser.c` reads the key.
3. RELAY: Parser injects key into active app history (e.g., `player_app/history.txt`).
4. TICK: App Module polls history buffer.
5. TRAIT/OP: Module calls a "Muscle" Op (e.g., `move_entity.+x`).
6. SOVEREIGNTY: Op updates Piece DNA (`.pd1`) or Mirror (`state.txt`).
7. MIRROR SYNC: Update flushed to `state.txt` for fast reading.
8. STAGE: `render_map.+x` reads mirrors and writes `view.txt`.
9. SYNC: `render_map.+x` appends to `frame_changed.txt` (pulse).
10. COMPOSITION: Parser substitutes variables and loads system state.
11. RENDER: Parser writes composite frame to `current_frame.txt`.

12. DISPLAY: Renderer (ASCII or GL) prints to user.

[TOOL 工] 7. Development Standards

CPU Safety (Mandatory)

Modules must follow the `fuzz_legacy_manager.c` pattern:

- **Signal Handling:** Explicit `SIGINT` handlers to prevent orphaned children.
- **Process Groups:** `setpgid(0, 0)` and group-killing on exit.
- **Fork/Exec:** Prefer `fork()/exec()/waitpid()` over `system()`.
- **Throttling:** Use `is_active_layout()` to sleep 100ms when not in focus.
- **Stat-First:** Only open files if `stat()` confirms they changed.

UI/Layout Patterns

- **Buttons:** NEVER manually write `[>]`. Use `<button label="X" onClick="KEY:n" />`.
- **CLI Input:** Use `<cli_io id="x" label="${var}" />` for text input. The parser handles buffering and masking.

---

8. How to Add to the Codebase

Creating a New App

1. Setup: `mkdir -p pieces/apps/<name>/{layouts,plugins/+x,pieces}`.
2. DNA: Create `<name>.pdl` with `app_id` and `layout_path`.
3. Logic: Implement `_module.c` using the CPU-Safe Template.
4. Wiring: Add a `<button href="...">` to `os.chtpm` or the loader.

Creating a New Project

1. Setup: `mkdir -p projects/<name>/{layouts,manager/+x,maps,pieces}`.
2. DNA: Create `project.pdl` with `project_id` and `entry_layout`.
3. Logic: Implement `_manager.c` to handle game state.

9. Current Status & Roadmap

"Stabilize the Core, Ignite the AI."

[OK 是] COMPLETED (April 2026)

Fondu Lifecycle Manager - Phase 1 Complete
- Install/uninstall projects
- Archive/restore functionality
- Ops registry system
- Auto-generated ops catalog
- Reference: `#.docs/fondu-investigation-report.txt`

op-ed (RMMP Editor) - 81% Feature Complete
- Piece selection (SPACE key)
- Undo (Ctrl+Z), Clear tile (Backspace)
- Create new map (ADD)
- Focus sync, PDL integration
- Save/load with roundtrip parity
- [WARN 警] Context menu (partial)
- [FAIL 错] Delete piece, Mirror sync (missing)
- Reference: `#.docs/op-ed-investigation-report.txt`

Windows Support - Working with Limitations
- GL-OS Desktop (OpenGL)
- Full CHTPM Pipeline
- [WARN 警] Arrow keys don't work in MSYS2/mintty (use number keys)
- [WARN 警] XInput controller support untested
- Reference: `pieces/buttons/WINDOWS-RUN-GUIDE.md`

Near-Term Priorities (Apr 2026)

1. Core Stabilization

- [ ] op-ed: Add delete piece functionality
- [ ] op-ed: Add mirror sync support
- [ ] Clean up project_routes.kvp (APP vs PROJECT topology)
- [ ] Populate or remove #.ref/ directory

2. PAL Scripting & AI

- [ ] PAL Mastery: Complete scripting engine
- [ ] AI-Recurse: Programmable bot behaviors
- [ ] Auto-battle systems

3. P2P Market & Trunking

- [ ] P2P-NET: Decentralized trading layer
- [ ] Entity trunking for high-turnover NPCs
- [ ] Market auctions and trading

4. Advanced Synthesis

- [ ] Chemistry/evolution systems
- [ ] Scripted contracts (legal/social logic)
- [ ] AI distillation for R&D

---

10. Future Roadmap

- **AI-LABS:** Knowledge distillation, local LLM training, and chat instance management.
- **LSR (Lunar Streetrace Raider):** A "Civ-Lite" simulation with AI-driven R&D and a lunar economy.
- **P2P-NET:** TPM-compliant blockchain, NFT items, and decentralized trading.
- **GL-OS (The Visual Shell):** High-fidelity OpenGL rendering shell mirroring text frames.
- **Perspective Views:** Real-time 1st person, 3rd person, and free-cam 3D renders of underlying 2D ASCII-OS projects.

- **Near-Term Targets:** Rapid implementation of 2D/3D visualizations for `op-ed` and `fuzz-ops`.
- **Concept:** GL-OS acts as a visual "theater" that bridges ASCII logic into a modern 3D viewport.
- **Kernel:** Long-horizon path to RISC-V/x86 kernel using Piece architecture.

---

11. Tools & Maintenance

- `./compile_all.sh`: Rebuilds all system and project modules.
- `./run_chtpm.sh`: Starts the environment pipeline.
- `./kill_all.sh`: Forcefully terminates all PMO processes.
- `#.tools/pmo_cpu_health.sh`: Monitors for runaway processes.
- `#.tools/fix_old_pieces.sh`: Migrates legacy pieces to `map_id` standards.

"Softness wins. The empty center of the flexbox holds ten thousand things."